



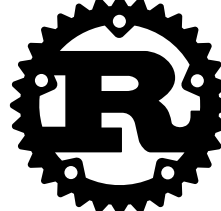
Pure Borrow

Linear Haskell Meets Rust-Style Borrowing

Yusuke Matsushita Kyoto University & MPI-SWS

Sept 15, 2026 @ Bristol PLRG

About Me

- ◆ **Yusuke Matsushita 松下 祐介**
 - ▶ July 15, 1996, Osaka
- ◆ **'15–'24 Bachelor, Master, Ph.D. at UTokyo**
 - ▶ **Computer science — Programming languages**
 - Supervised by Prof. Naoki Kobayashi
- ◆ **'25– Assist. Prof. at Hakubi, KyotoU**
 - ▶ Focus on Rust 
 - ▶ '26– Visiting scientist at MPI-SWS
 - Germany, Prof. Derek Dreyer



15th/Graduate School of Informatics
Yusuke MATSUSHITA
Assistant Professor

Pioneering the next
generation of software
development technology with
the power of logic

— Exploring a New Age of
Software Development Springing
from Rust



About Me

Google Scholar

SIGN IN



Yusuke Matsushita

FOLLOW

Kyoto University

Verified email at fos.kuis.kyoto-u.ac.jp - [Homepage](#)

Computer Science Programming Languages Machine Learning

TITLE

CITED BY

YEAR

RustHorn: CHC-based Verification for Rust Programs

Y Matsushita, T Tsukada, N Kobayashi

ACM Transactions on Programming Languages and Systems (TOPLAS) 43 (4), 1-54

109

2021

RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code

Y Matsushita, X Denis, JH Jourdan, D Dreyer

Proceedings of the 43rd ACM SIGPLAN Conference on Programming Language ...

95

2022

RustHorn: CHC-Based Verification for Rust Programs

Y Matsushita, T Tsukada, N Kobayashi

European Symposium on Programming (ESOP), 484-514

65

2020

Nola: Later-Free Ghost State for Verifying Termination in Iris

Y Matsushita, T Tsukada

Proceedings of the ACM on Programming Languages 9 (PLDI)

8

2025

Borrowable Fractional Ownership Types for Verification

T Nakayama, Y Matsushita, K Sakayori, R Sato, N Kobayashi

International Conference on Verification, Model Checking, and Abstract ...

7

2024

GET MY OWN PROFILE

Cited by

	All	Since 2021
Citations	301	294
h-index	5	5
i10-index	3	3

70

35

0

2020

2021

2022

2023

2024

2025

2026

Public access

VIEW ALL

0 articles

6 articles

not available

available

3

About Me



PLDI Review Committee PLDI Research Papers



Benjamin Mariano
The University of Texas at Austin, Texas, USA
United States



Guido Martínez
Microsoft Research
United States



Yusuke Matsushita
Kyoto University
Japan



Jedidiah McClurg
Cornell University
United States



Roland Meyer
TU Braunschweig
Germany



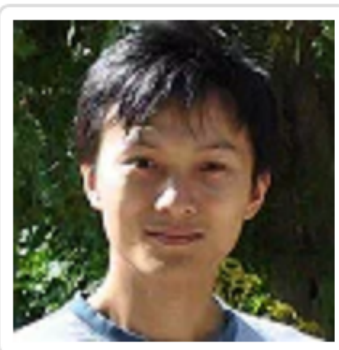
Mae Milano
Princeton University
United States



Program Committee POPL



Yannick Zakowski
Inria
France



Yingfei Xiong
Peking University
China



Youyou Cong
Institute of Science Tokyo
Japan



Yu Feng
University of California at Santa Barbara
United States



Yu-Fang Chen
Academia Sinica
Taiwan



Yusuke Matsushita
Kyoto University
Japan

Up to Now

High Schooler

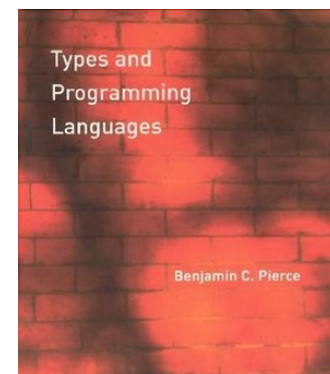
Undergrad

Grad

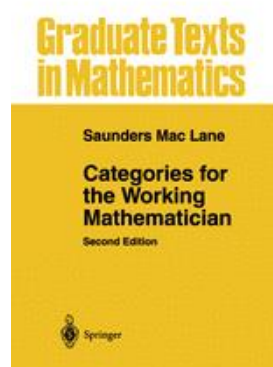
Now

 **Haskell**

λ -calculus Monad



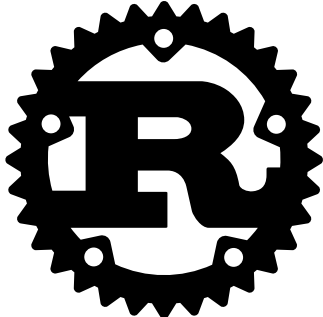
Type theory



Category theory



PL

 **Rust**

*Fast like C/C++,
Safe like Haskell?*



Study PL!



RustHorn

ESOP '20 & TOPLAS '21

Rust $\rightarrow$ Functional



RustHornBelt

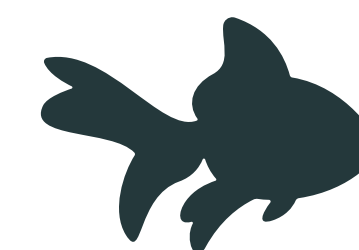
PLDI '22

Logic for RustHorn



Nola

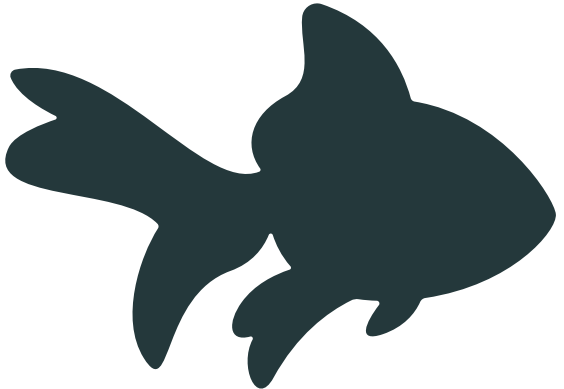
PLDI '25



Pure Borrow

PLDI '26

Haskell Meets Rust!

Pure Borrow 

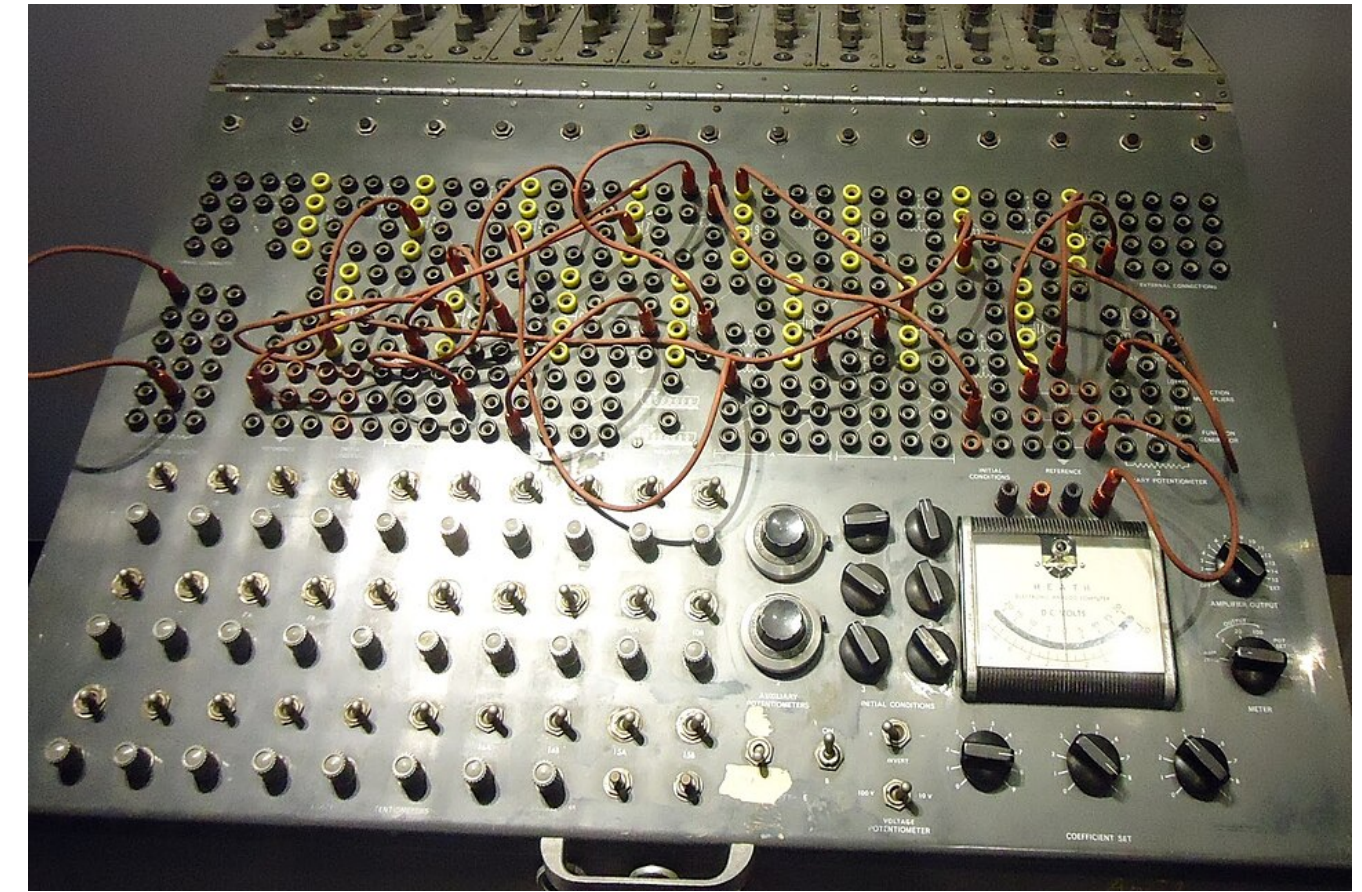
***Rust-Style Borrowing in
Linear Haskell with Purity***

Introduction

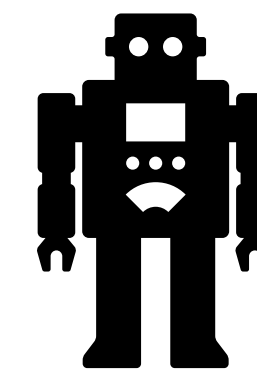
Software



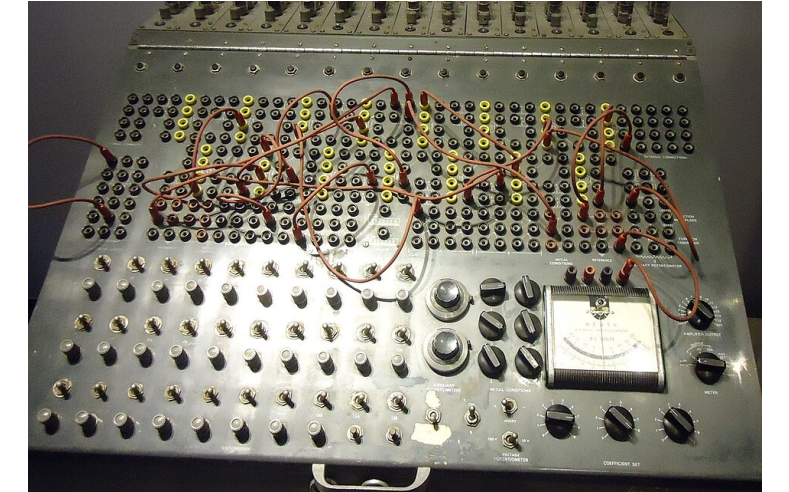
Software



Software embodies *human ideals*
on *finite physical machines*



Two Major Paradigms of Programming



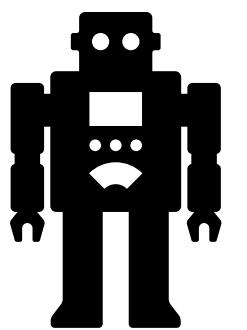
Functional

High-level abstraction

Well-behaved

Safe & Predictable

Imperative

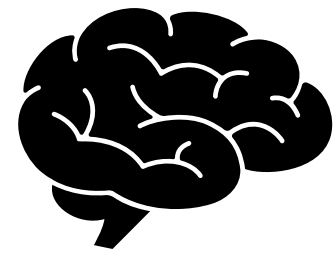


Low-level control

Well-tuned

Performative

Functional, with Purity



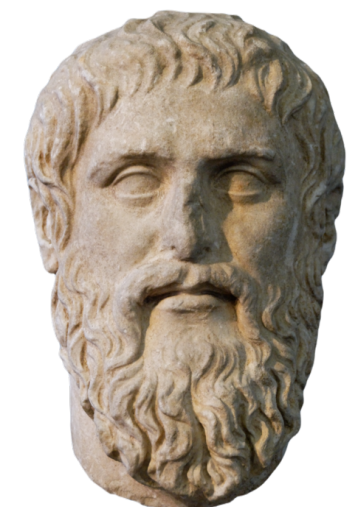
 **Haskell** '90



λ -calculus '36

Purity

***Same result
regardless of timing***



Well-behaved, esp. *parallelism*

Purity = Referential Transparency

Super Well-behaved!



Deterministic

*Same result
across executions*

$(t, t) \equiv \text{let } x = t \text{ in } (x, x)$

✓ **Predictable**

✓ **Equations**

Order-insensitive

*Commutates with
other computations*

$(t, u) \equiv \text{let } x = u \text{ in } (t, x)$

Parallelism, Lazy evaluation, ...

Reasoning & Optimization

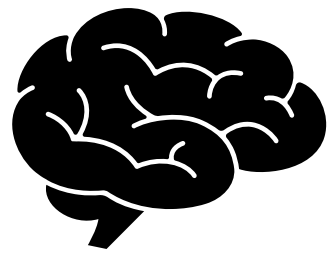
e.g. $\text{map } (g \circ f) \equiv \text{map } g \circ \text{map } f$

Functional, More Imperative



Linear Haskell *Bernardy+ '18*

◀ ⚡ **Mutate!** **Linear Types** '90–



» **Haskell** '90

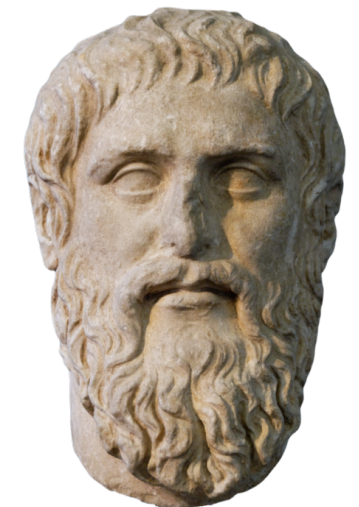


λ -calculus '36

Performance?

Purity

**Same result
regardless of timing**



Well-behaved, esp. *parallelism*

- Original Haskell **Persistent** data $\rightarrow$ **Copy**

$$\text{Vec } a \xrightarrow{\quad} \text{Int} \rightarrow (a \rightarrow a) \rightarrow \text{Vec } a$$

$$[0, 1, 2, \dots, 999] \quad 0 \quad (+\ 7) \quad [7, 1, 2, \dots, 999] \quad \text{Pure!}$$



- Linear Haskell **Linear** $\rightarrow$ **Mutate** in place!

Linear arrow $\multimap$ Argument is used exactly **once**

$$\underline{\text{Vec } a} \multimap \text{Int} \rightarrow (a \multimap a) \multimap \underline{\text{Vec } a}$$

$$\underline{[0, 1, 2, \dots, 999]} \quad 0 \quad (+\ 7) \quad [7, 1, 2, \dots, 999] \quad \text{Pure!}$$

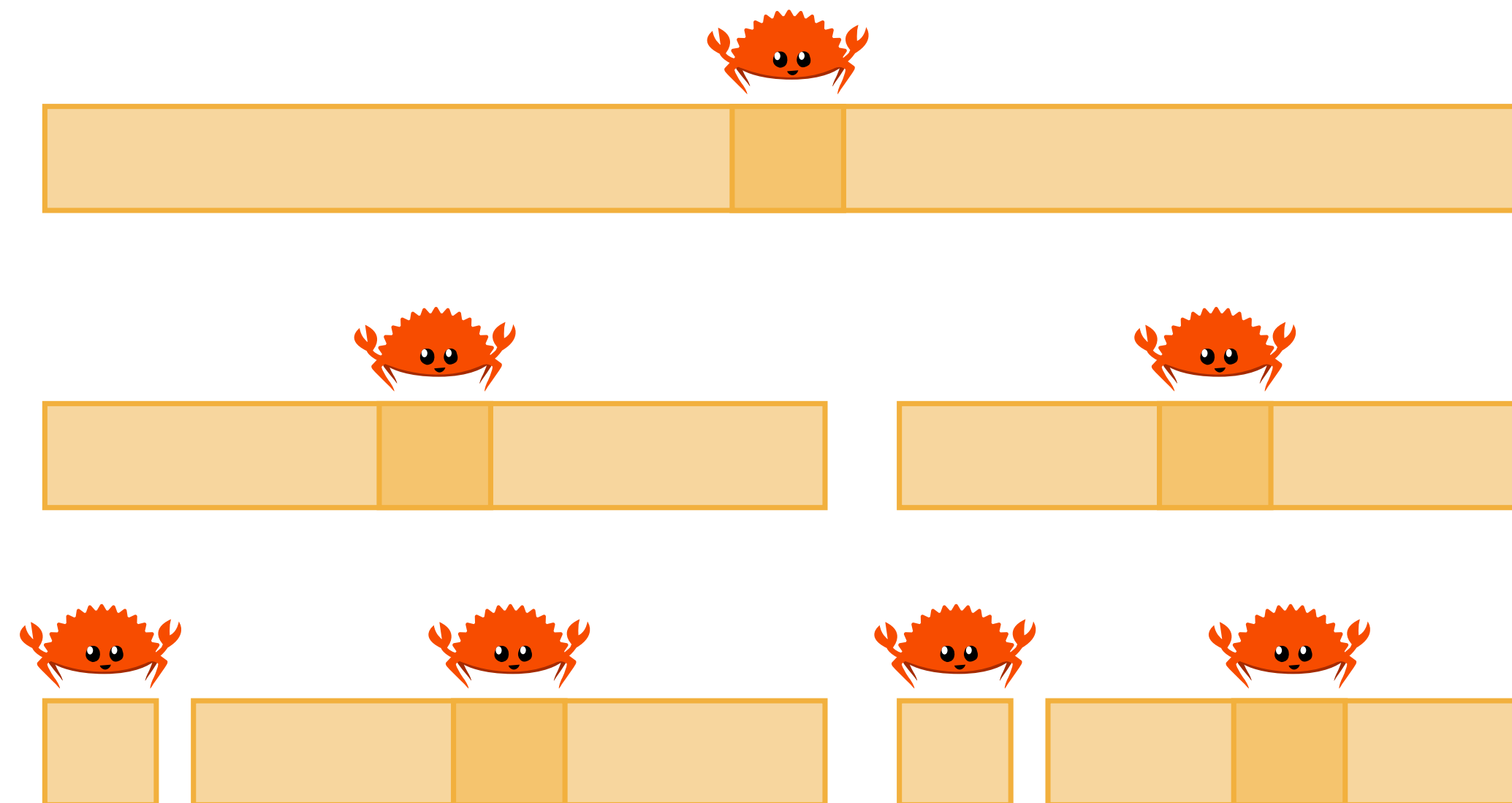
No aliases! 

Direct ownership passing

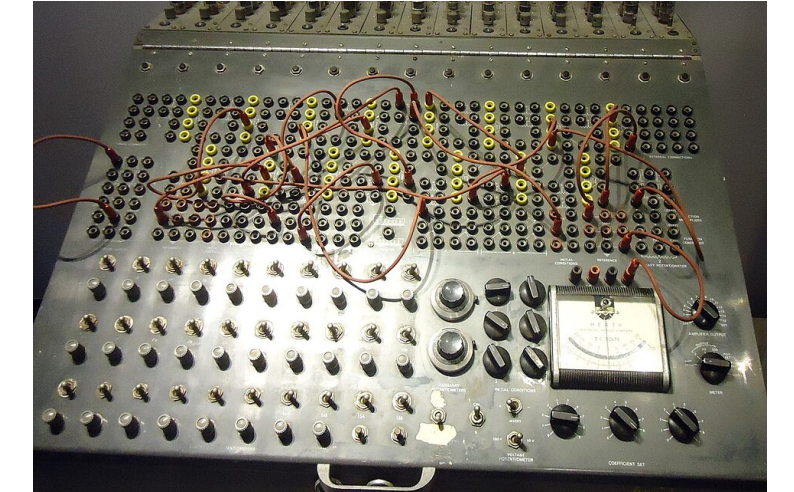
Challenge

Separate mutations to disjoint subregions

E.g., **Parallel quicksort**



Two Major Paradigms of Programming



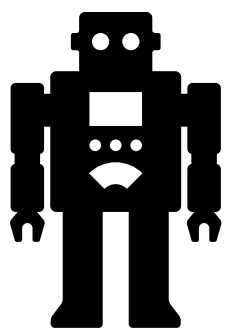
Functional

High-level abstraction

Well-behaved

Safe & Predictable

Imperative

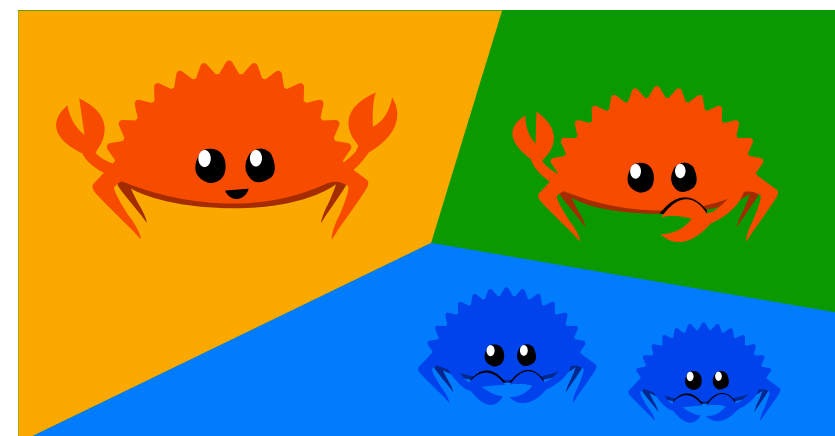
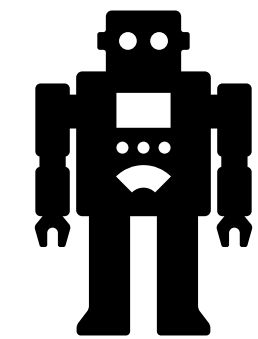
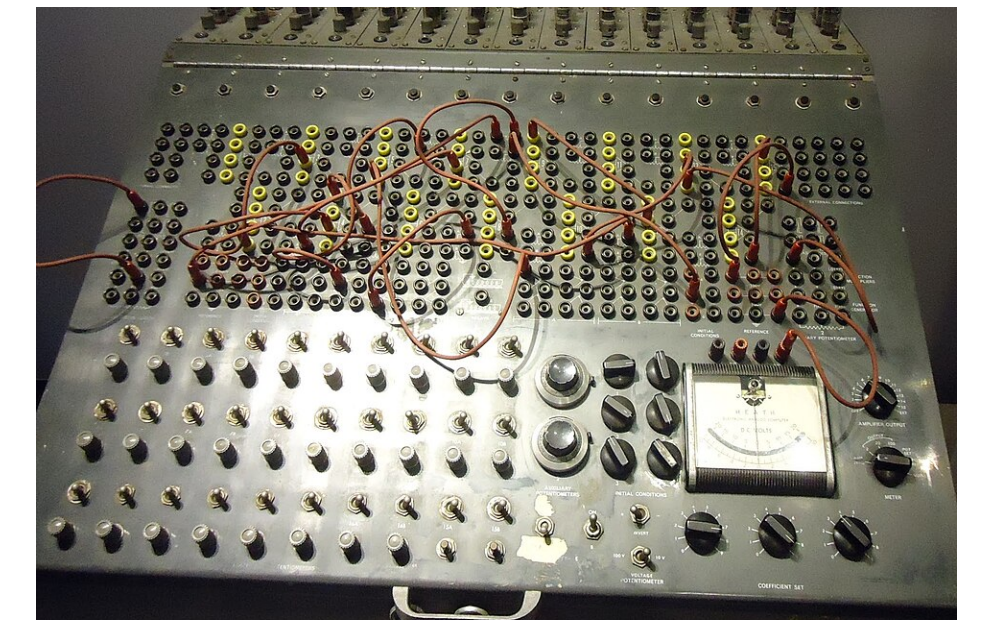


Low-level control

Well-tuned

Performative

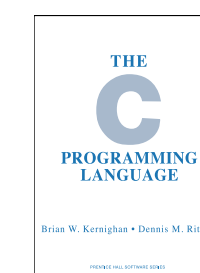
Imperative, More Functional



Ownership & Region Types '94–



Memory corruption, Race condition, ...



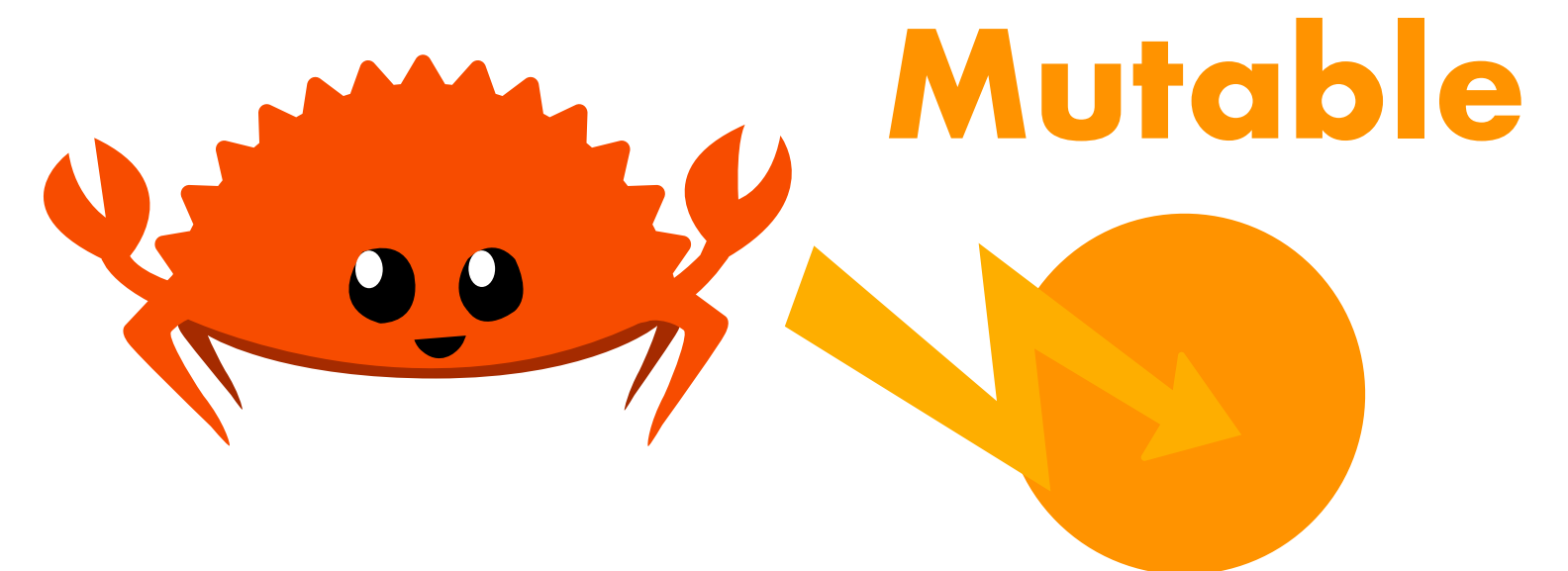
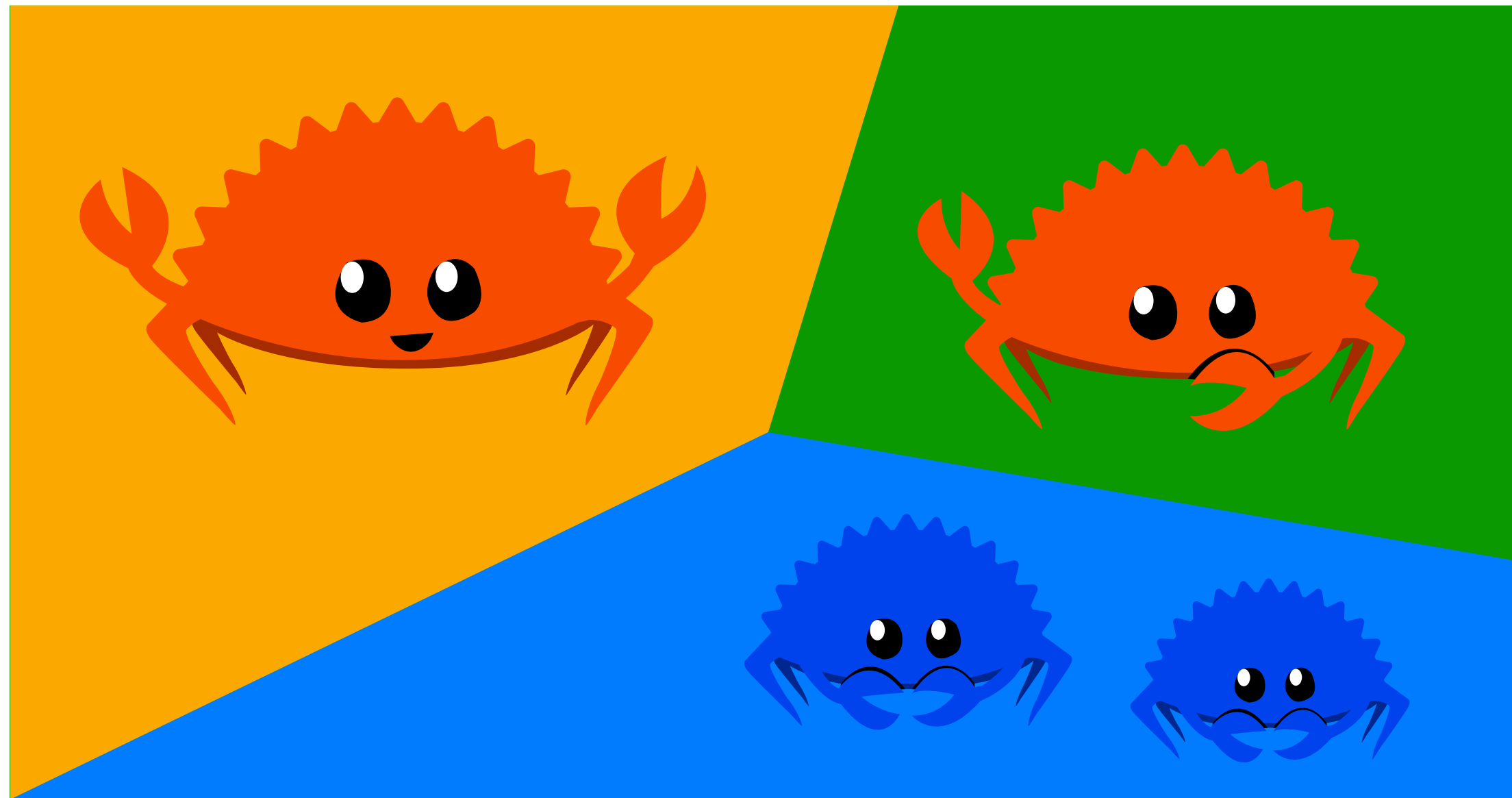
'72



'85

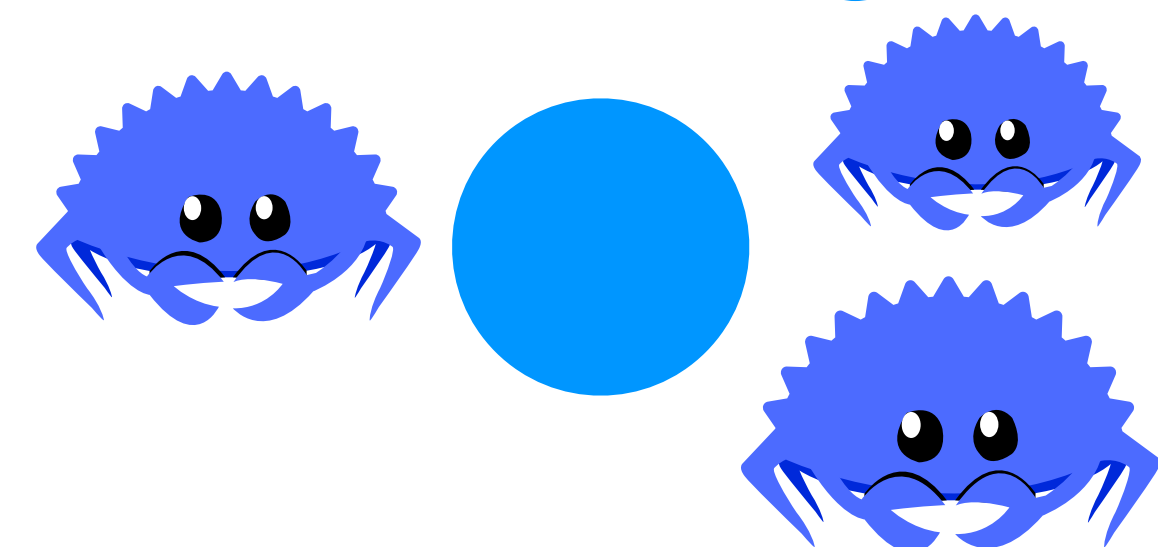
Ownership

Separate **Mutable** State



XOR

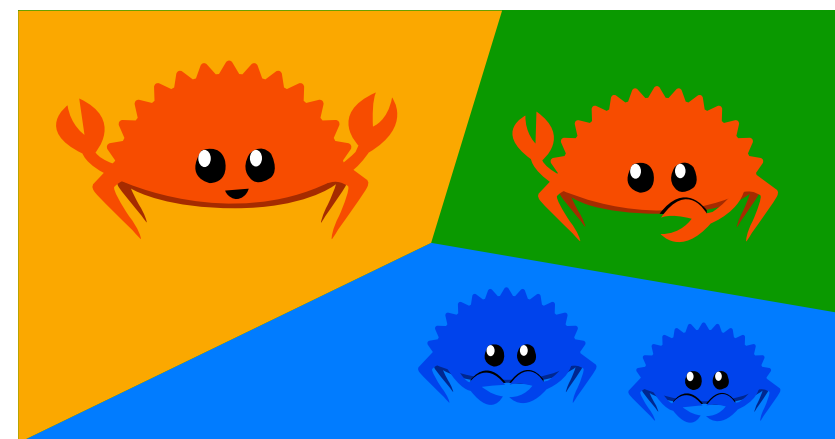
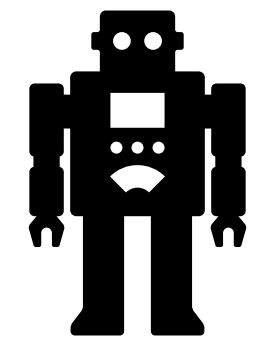
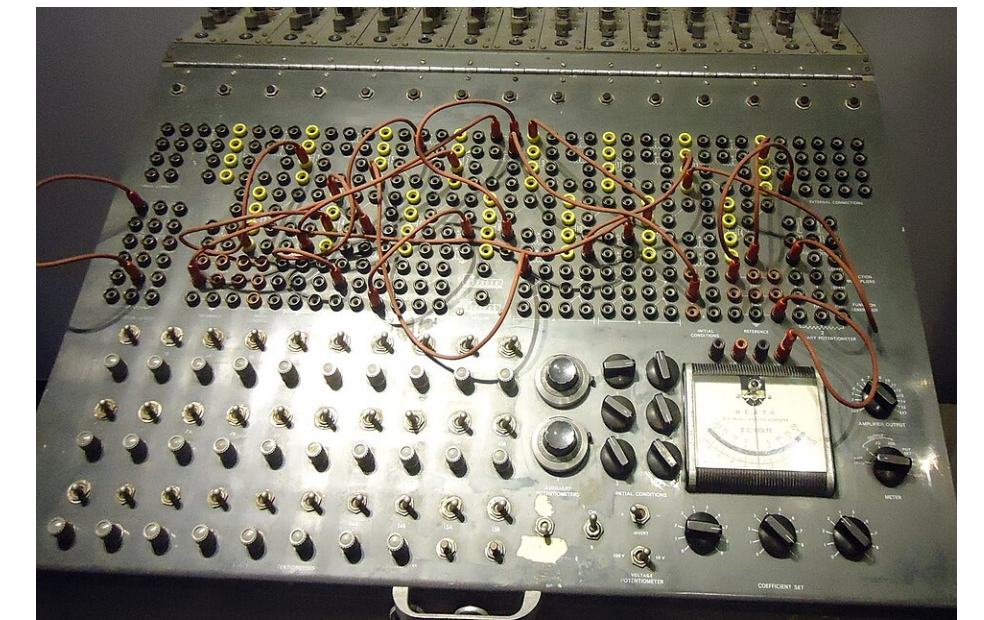
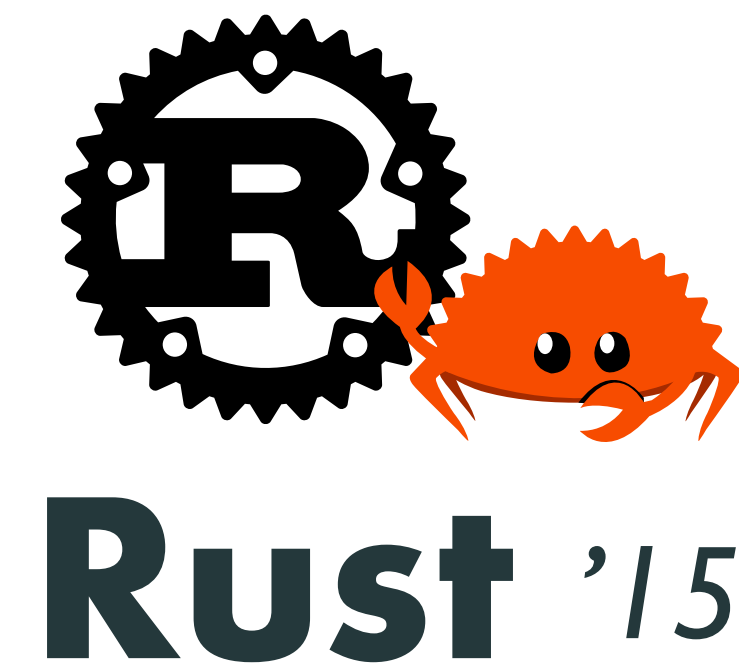
Aliasing



→ ***No dangerous interference between computations***

Imperative, More Functional with Borrows

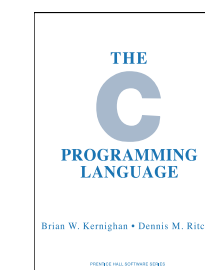
Borrow ↻ **Non-local!** ✈️
Separate ownership by **time**
Mutation w/o ownership passing



Ownership & Region Types '94—



Memory corruption, Race condition, ...

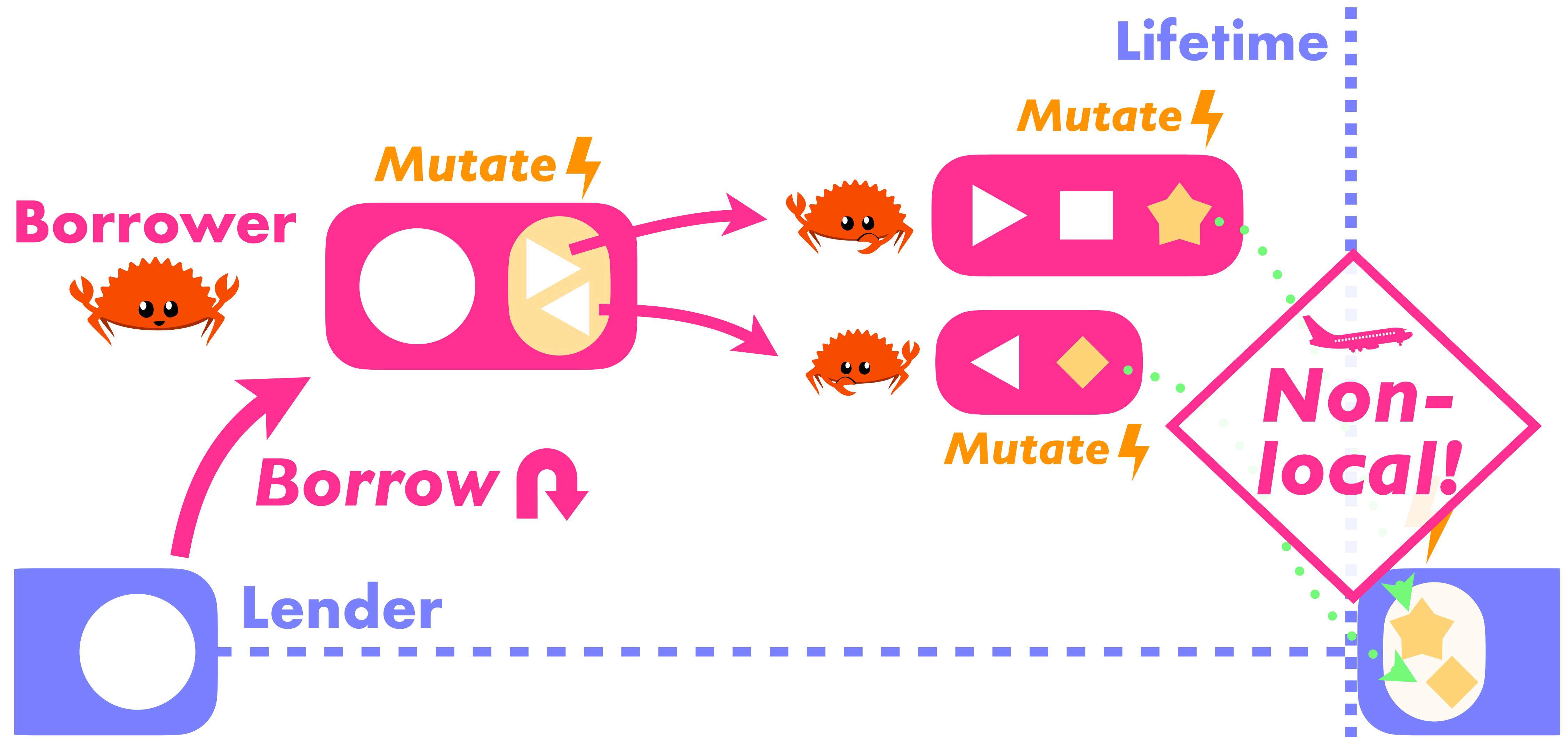


'72



'85

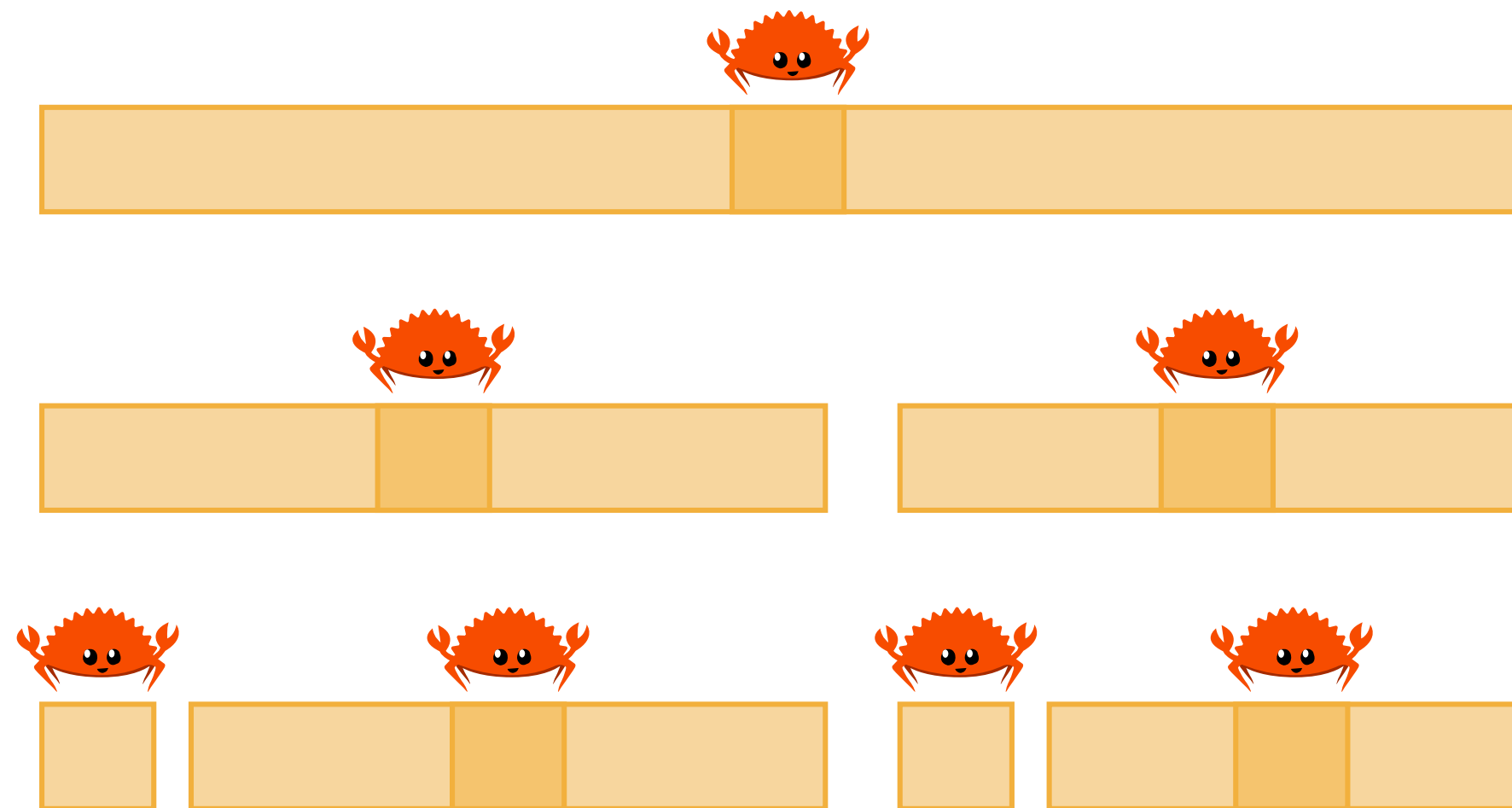
Rust-style Borrow



Borrowing Solved! But, ...

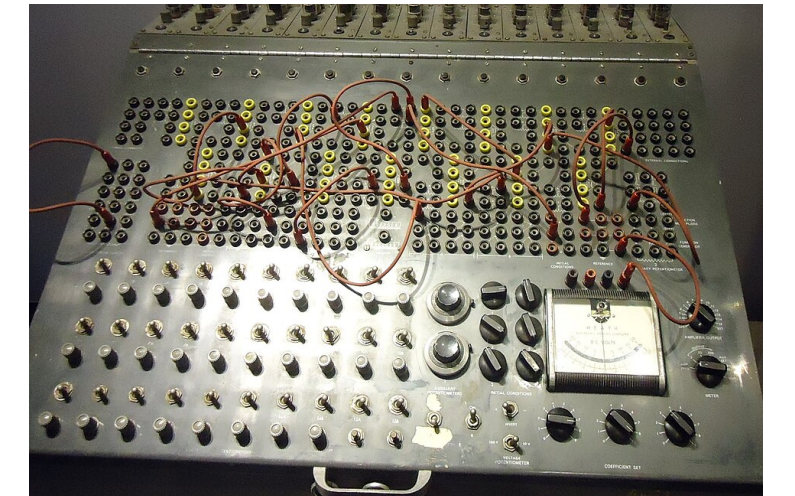
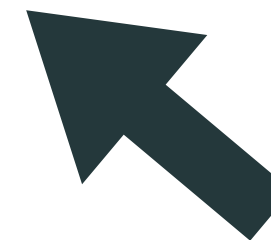
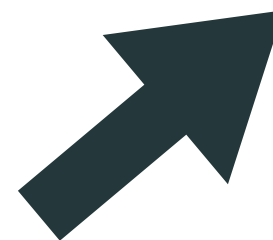
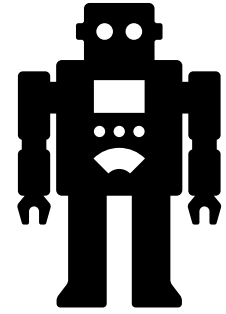
✓ **Borrow** *Separate mutations to disjoint subregions*

E.g., **Parallel** quicksort



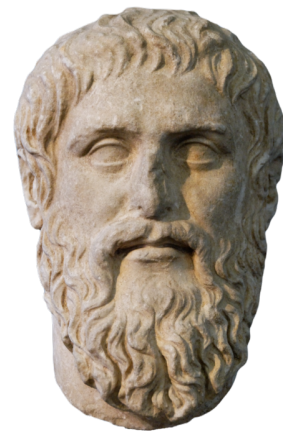
Purity?

Challenge



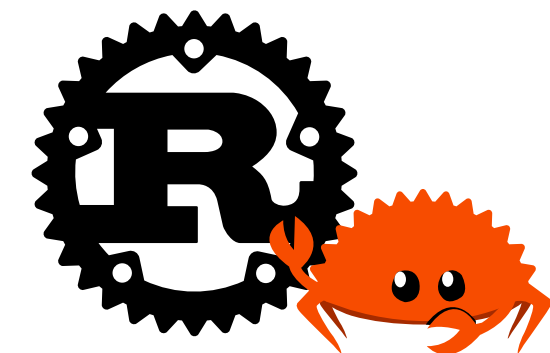
Purity

*Same result
regardless of timing*

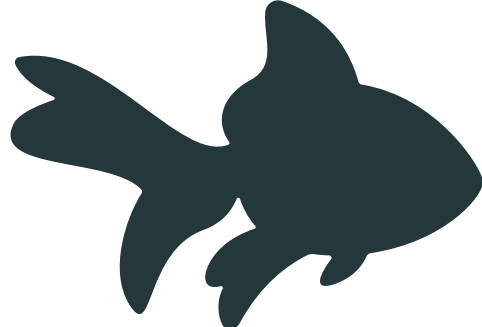


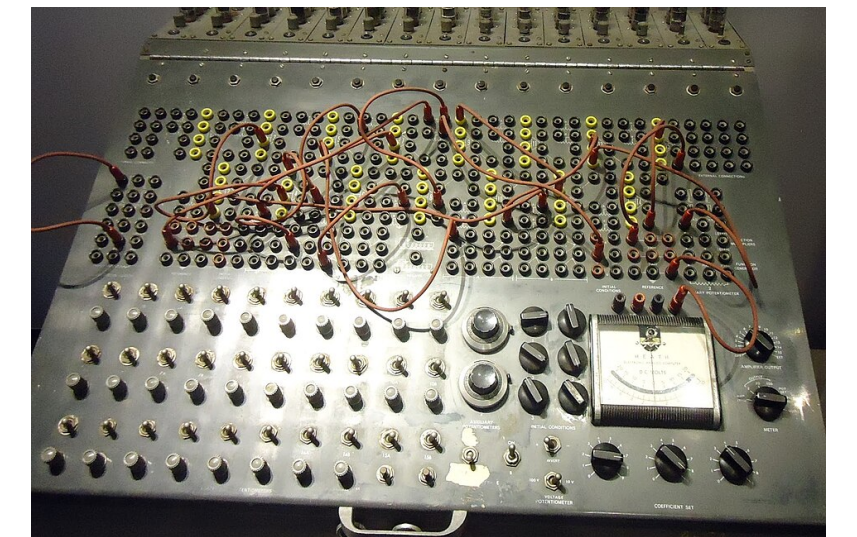
Borrow ↻ **Non-local!** ✈️
Separate ownership by time
Mutation w/o ownership passing

 **Linear Haskell**

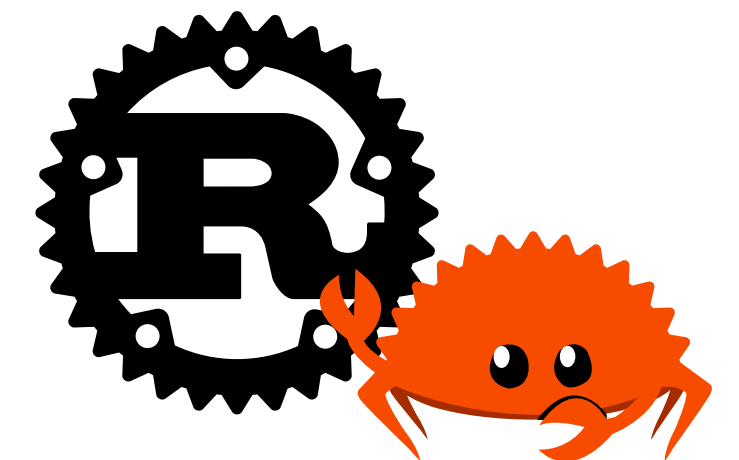
 **Rust**



Pure Borrow 



***Rust-Style **Borrowing** in
Linear Haskell with **Purity*****



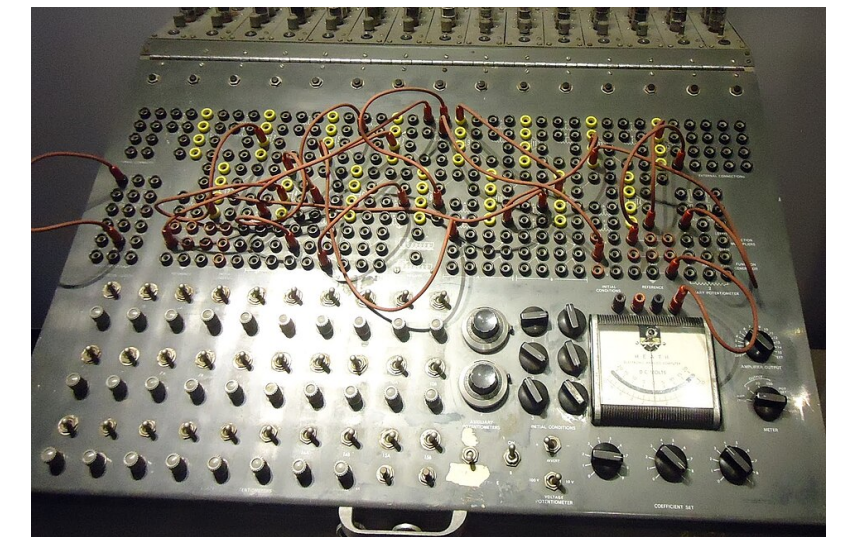
High-Level Comparison

	Pure 	Mutate  Borrow 	Parallel	Leak-free
Pure Only	●	○	●	
ST Monad	●	●	○	○
Rust	○	●	●	○
Basic Linear Haskell	●	●	●	●
Pure Borrow	●	●	●	●



Pure Borrow

*Rust-Style **Borrowing** in
Linear Haskell with **Purity***



- ✦ **Pure & Mutate & Parallelize** *Parallel quicksort*
- ✦ **Just APIs** *Rich features of Haskell*
- ✦ **Metatheory: Why Pure** WIP *Histories for borrows*

This Talk



- ✦ **Overview**
- ✦ **Case Study**
- ✦ **Metatheory**

- ✦ **Related Work**
- ✦ **Conclusion**

Overview

Preliminary Linearity in Linear Haskell

data Ur a **where**

$Ur :: a \rightarrow Ur\ a$

Unrestricted ω

Default

Linear 1

Opt-in

$lin :: (Lin \Rightarrow Ur\ a) \multimap Ur\ a$ cf. $runST :: (\forall s. ST^s\ a) \rightarrow a$

class Lin **Linearity witness** *Spiwack+ '22*

Used arbitrarily finite times But never in Ur

Preliminary Mutable State in Linear Haskell

Vector **data** Vec a

newVec :: Lin $\Rightarrow$ [a] $\multimap$ Vec a



Linear!

freeVec :: Vec a $\multimap$ [a]

class Movable a **where**

move :: a $\multimap$ Ur a

Before Pure Borrow

modifyAt' :: Vec a $\multimap$
Int $\rightarrow$ (a $\multimap$ a) $\multimap$ Vec a

readAt' :: Movable a $\Rightarrow$
Vec a $\multimap$ Int $\rightarrow$ (Ur a, Vec a)

Direct ownership passing!

Preliminary Example of Basic Linear Haskell

example :: **Ur** (Int, [Int])

example = **lin do**[◦]

$v \leftarrow \text{newVec } [0,1,2]$

v $\leftarrow$ **modifyAt'** v 0 (+ 3); v $\leftarrow$ **modifyAt'** v 2 (+ 5)

v $\leftarrow$ **modifyAt'** v 0 ($\times$ 4)

Direct ownership passing!

(**Ur** n, v) $\leftarrow$ **readAt'** 0 v

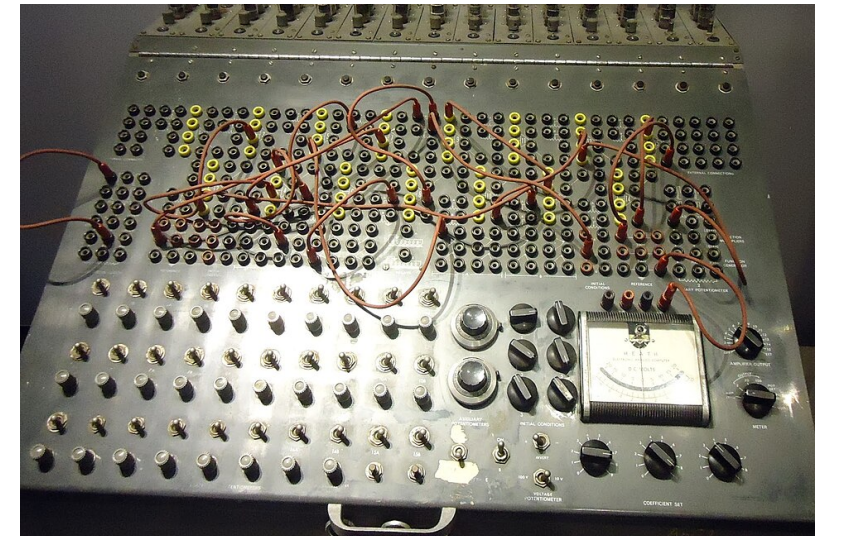
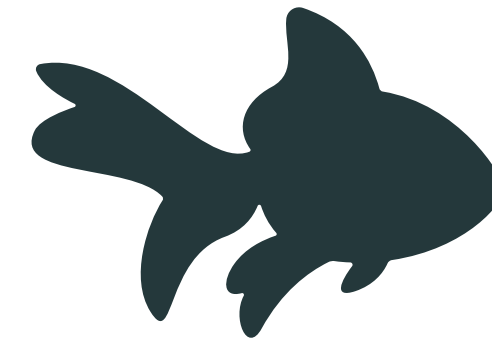
move (n, freeVec v) **Ur** (12, [12, 1, 7])

do[◦] $p \leftarrow t; c \equiv$

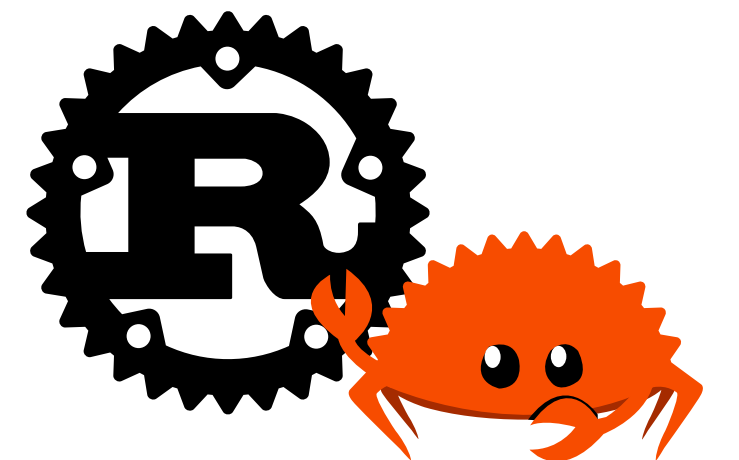
let ! $p = t$ **in do**[◦] c



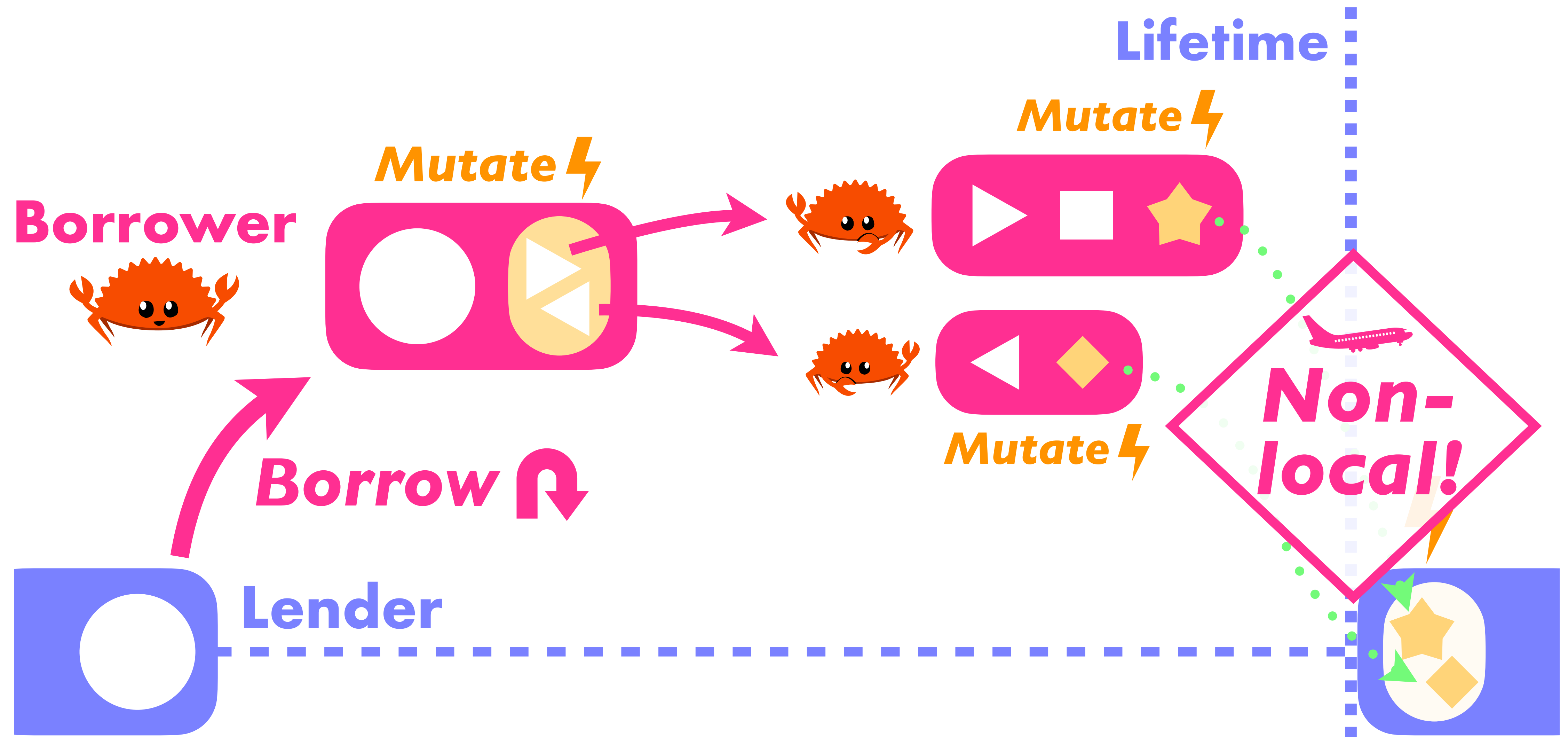
Pure Borrow



*Rust-Style **Borrowing** in
Linear Haskell with **Purity***

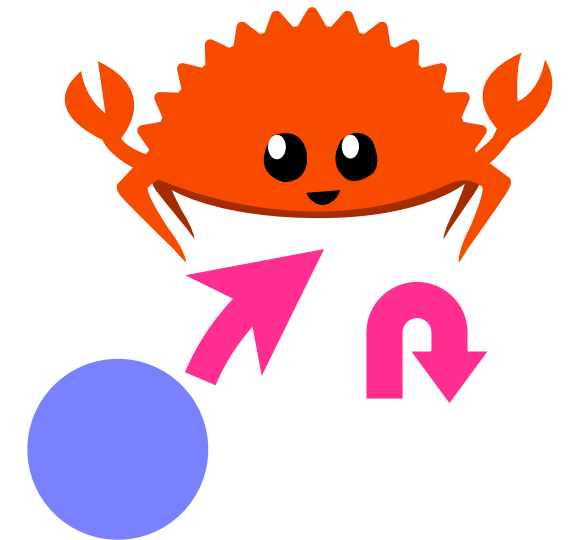


Recap Rust-Style Borrow

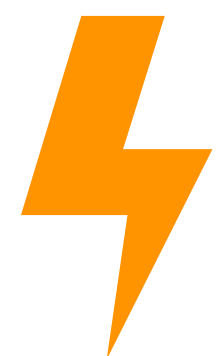


Core APIs

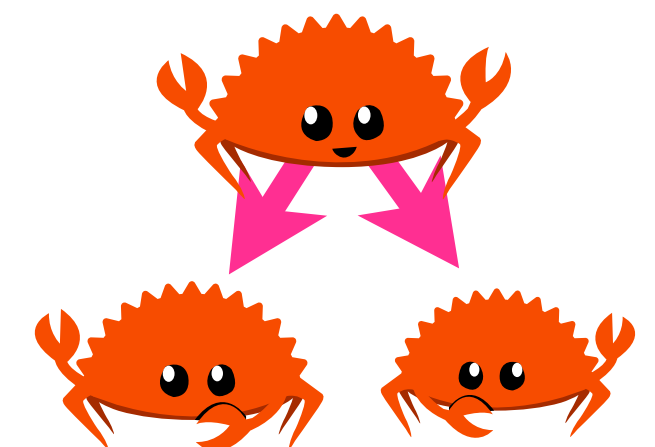
Borrow **Mutable borrower** **Lender**
 $\text{borrow} :: \text{Lin} \Rightarrow a \multimap (\text{Mut}^\alpha a, \text{Lend}^\alpha a)$
newtype of a



Mutate $\text{modifyAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap$
 $\text{Int} \rightarrow (a \multimap a) \multimap \text{BO}^\alpha (\text{Mut}^\alpha (\text{Vec } a))$
Monad



Split $\text{splitAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap$
 $\text{Int} \rightarrow (\text{Mut}^\alpha (\text{Vec } a), \text{Mut}^\alpha (\text{Vec } a))$



Reclaim $\text{reclaim} :: \text{End}^\alpha \xRightarrow{\alpha \text{ has ended}} \text{Lend}^\alpha a \multimap a$



BO Monad

BO^α a **Computation permitted only before α ends**

Borrower accesses BO^α $< \alpha \text{ ends} \leq$ **Lender reclaim** End^α

Pure $\text{runBO} :: \text{Lin} \Rightarrow (\forall \alpha. \text{BO}^\alpha (\text{End}^\alpha \Rightarrow a)) \multimap a$
Rank-2 poly cf. $\text{runST} :: (\forall s. \text{ST}^s a) \rightarrow a$



Linearity ensures commutativity

Parallel $\text{parBO} :: \text{BO}^\alpha a \multimap \text{BO}^\alpha b \multimap \text{BO}^\alpha (a, b)$

cf. *ST is not parallelizable*

Example

```
example :: Ur [Int]
example = lin doo
  v ← newVec [0, 1, 2]
  runBO do
    let !(mv, lv) = borrow v
    mut_example mv
    pure $ move $
      freeVec $ reclaim lv
    Ur [12, 1, 7]
```

```
mut_example ::
  Mutα (Vec Int) → BOα ()
mut_example mv = do
  mv ← modifyAt mv 0 (+ 3)
  mv ← modifyAt mv 2 (+ 5)
  mv ← modifyAt mv 0 (× 4)
  pure $ consume mv

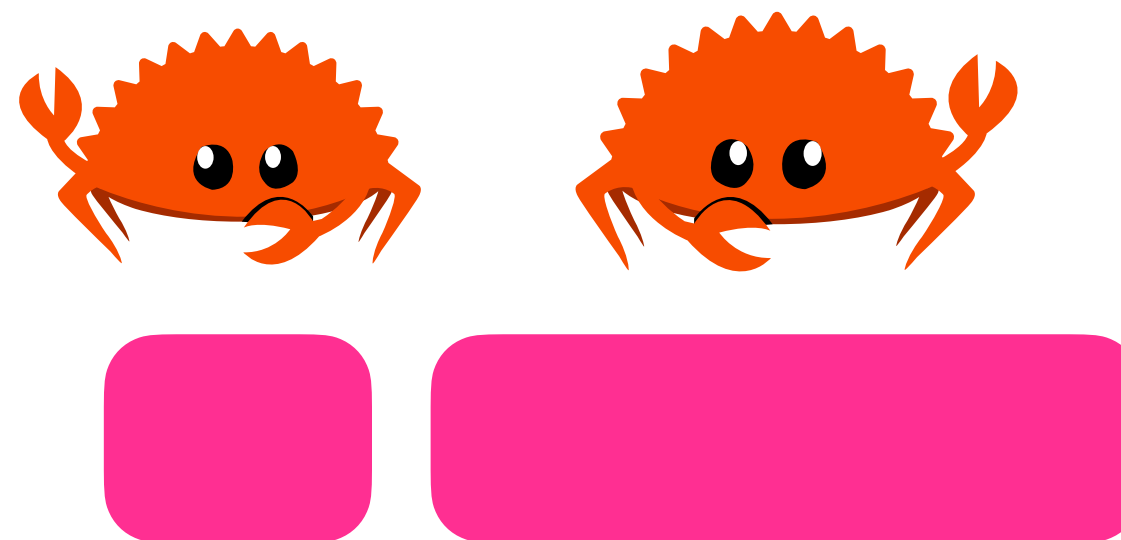
class Consumable a where
  consume :: a → ()
```

Example with Parallelization

```
example :: Ur [Int]
example = lin doo
  v ← newVec [0, 1, 2]
  runBO do
    let !(mv, lv) = borrow v
    mut_example mv
  pure $ move $
    freeVec $ reclaim lv

Ur [12, 1, 7]
```

```
mut_example mv =
  let !(mv0, mv1) = splitAt mv 1 in
  consume <$> parBO
    (do mv0 ← modifyAt mv0 0 (+ 3)
     modifyAt mv0 0 (× 4))
    (modifyAt mv1 1 (+ 5))
```

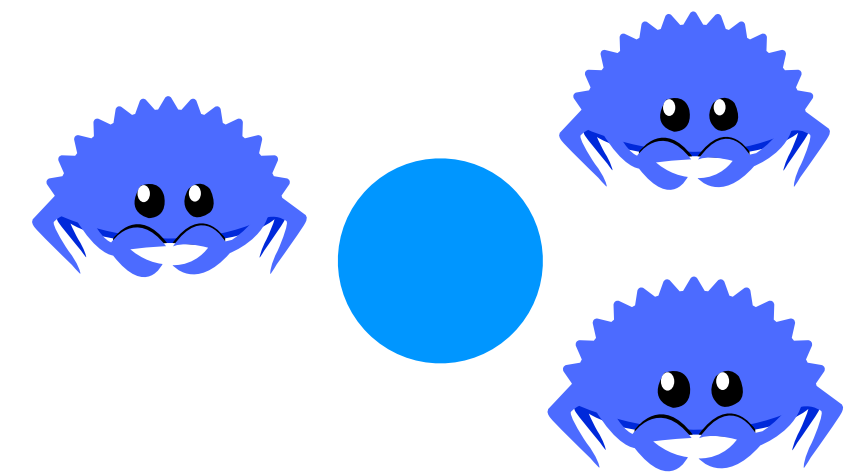


Parallelize!

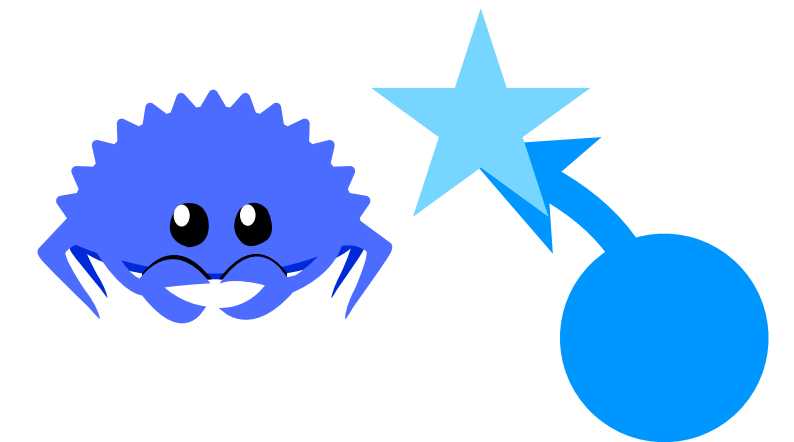
Share

Shared borrower

Share $\text{share} :: \text{Mut}^{\alpha} a \multimap \text{Ur} (\text{Share}^{\alpha} a)$

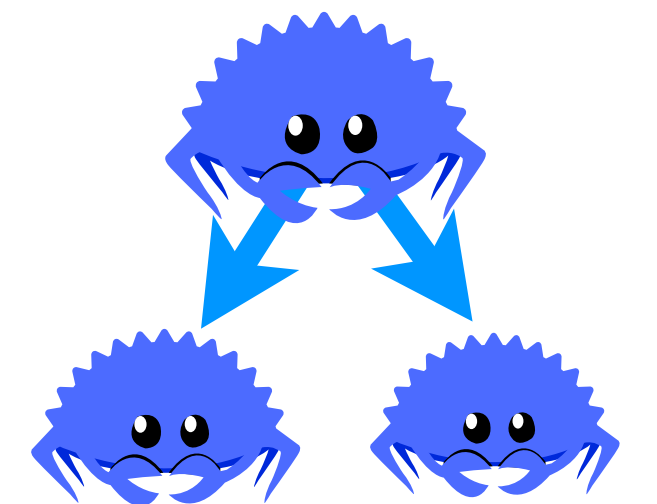


Read $\text{copyAt} :: \text{Copyable } a \Rightarrow$
 $\text{Share}^{\alpha} (\text{Vec } a) \rightarrow \text{Int} \rightarrow \text{BO}^{\alpha} (\text{Ur } a)$



class Copyable a **where**
 $\text{copy} :: \text{Share}^{\alpha} a \rightarrow a$

Split $\text{splitAt} :: \text{Share}^{\alpha} (\text{Vec } a) \multimap$
 $\text{Int} \rightarrow (\text{Share}^{\alpha} (\text{Vec } a), \text{Share}^{\alpha} (\text{Vec } a))$



Example

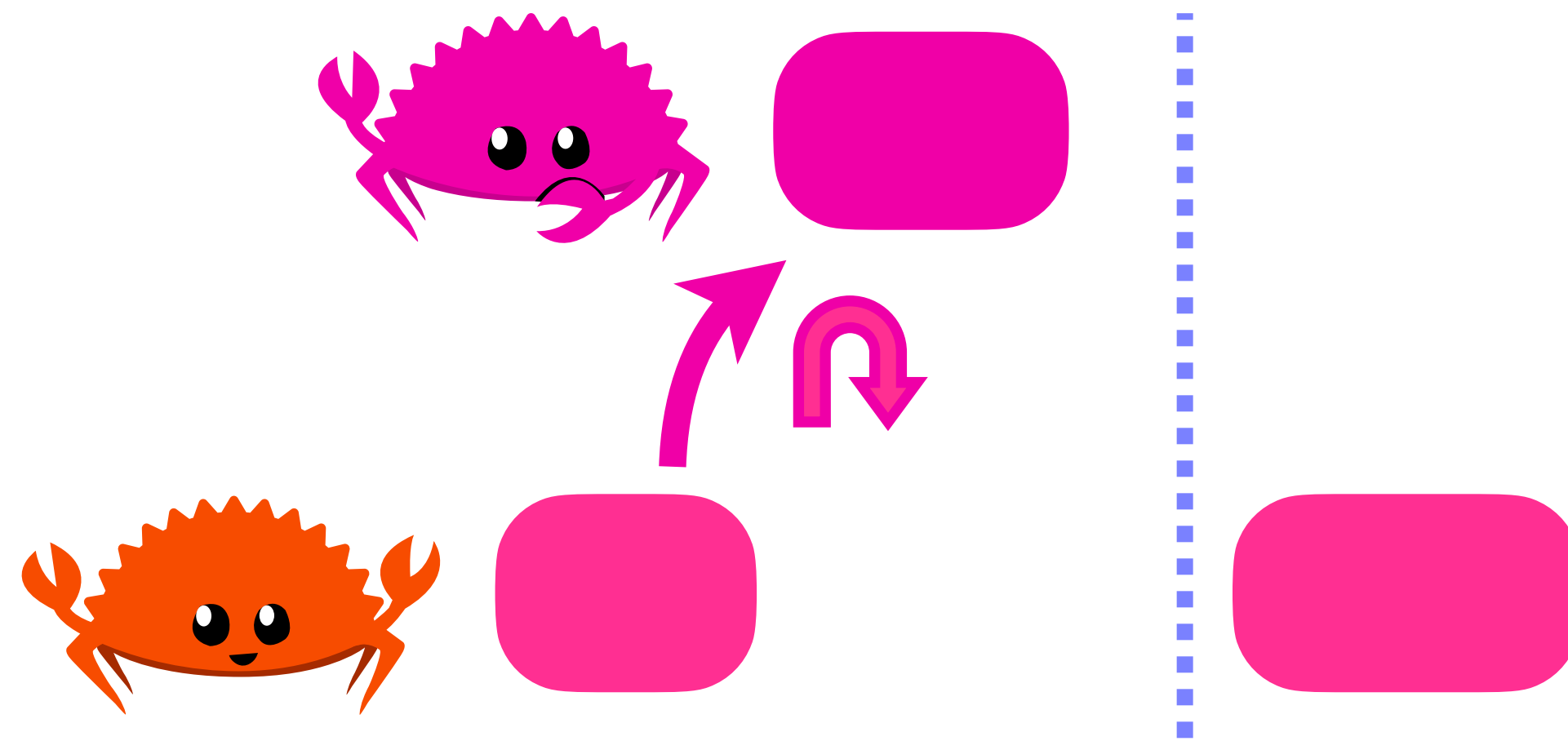
```
example :: Ur (Int, [Int])
example = lin doo
  v ← newVec [0, 1, 2]
  runBO do
    let !(mv, lv) = borrow v
    n ← mut_example mv
    pure $ move $
      (n, freeVec $ reclaim lv)

    Ur (12, [12, 1, 7])
```

```
mut_example :: Mutα (Vec Int) → BOα Int
mut_example mv = do
  let !(mv0, mv1) = splitAt mv 1
  (Ur n, mv1) ← parBO Can't keep it?
    (do mv0 ← modifyAt mv0 0 (+ 3)
     modifyAt mv0 0 (× 4)
     let !(Ur sv) = share mv0
     copyAt sv 0)
    (modifyAt mv1 1 (+ 5))
  let () = consume mv1; pure n
```

Reborrow

Reborrow



reborrowing $:: \text{Mut}^\alpha a \multimap$

$(\forall \beta. \text{Mut}^{\beta \wedge \alpha} a \multimap \text{BO}^{\beta \wedge \alpha'} r) \multimap$

$\text{BO}^{\alpha'} (r, \underline{\text{Mut}^\alpha a})$ **Lifetime
intersection**

Example

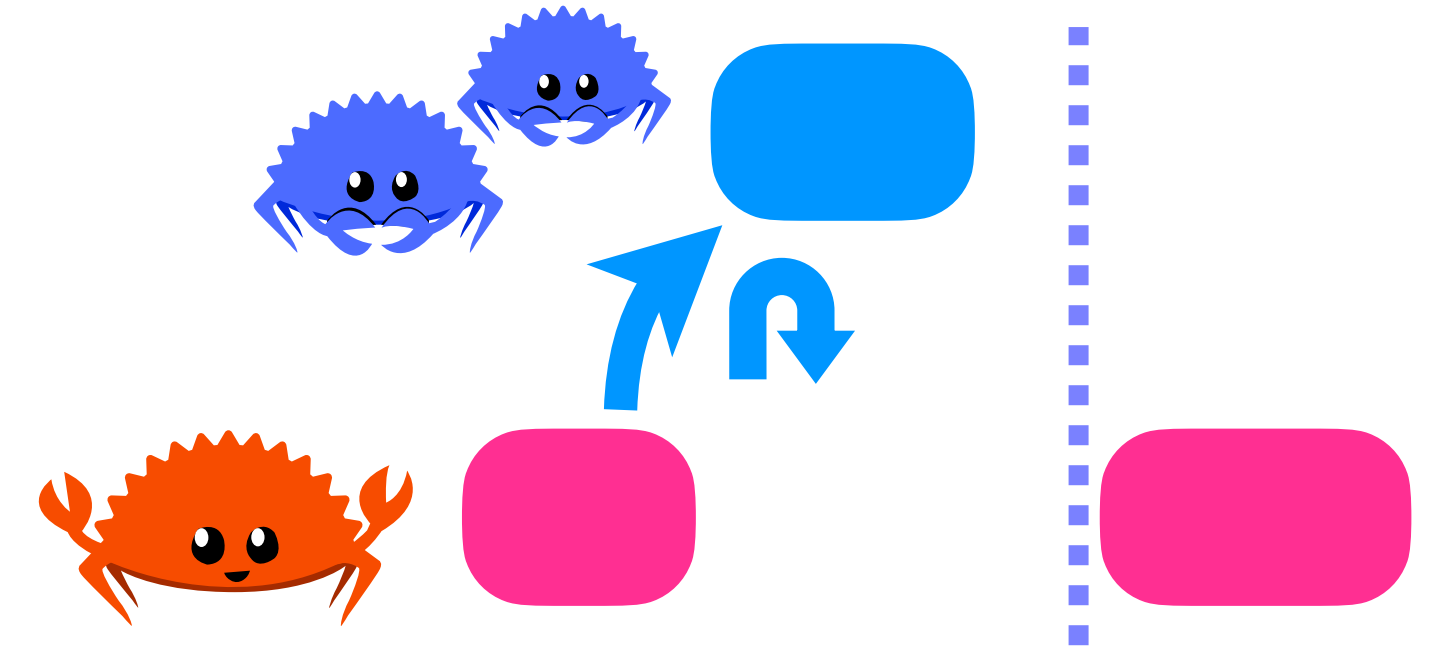
```
example :: Ur (Int, [Int])
example = lin doo
  v ← newVec [0, 1, 2]
  runBO do
    let !(mv, lv) = borrow v
    n ← mut_example mv
    pure $ move $
      (n, freeVec $ reclaim lv)
    Ur (12, [12, 1, 7])
```

```
mut_example :: Mutα (Vec Int) → BOα Int
mut_example mv = do
  ((), mv) ← reborrowing mv λmv →
    let !(mv0, mv1) = splitAt mv 1
    consume <$> parBO
      (do mv0 ← modifyAt mv0 0 (+ 3)
        modifyAt mv0 0 (× 4))
      (modifyAt mv1 1 (+ 5))
  let !(Ur sv) = share mv; copyAt sv 0
```

Temporary Sharing

Temporarily share

$\text{sharing} :: \text{Mut}^\alpha a \multimap$
 $(\forall \beta. \text{Share}^{\beta \wedge \alpha} a \rightarrow \text{BO}^{\beta \wedge \alpha'} r) \multimap$
 $\text{BO}^{\alpha'} (r, \text{Mut}^\alpha a)$



$\text{sharing } m \ k =$
 $\text{reborrowing } m \ \lambda m \rightarrow$
 $\text{let } !(Ur \ s) = \text{share } m \text{ in } k \ s$

Read from Mut

$\text{copyAtMut} :: (\text{Copyable } a, \beta \leq \alpha) \Rightarrow$
 $\text{Mut}^\alpha (\text{Vector } a) \rightarrow \text{Int} \rightarrow$
 $\text{BO}^\beta (Ur \ a, \text{Mut}^\alpha (\text{Vector } a))$

$\text{copyAtMut } i \ mv =$
 $\text{sharing } mv \ \$ \ \text{copyAt } i$

Remark Internal Implementation

Just unsafe coercions → Efficient

borrow = unsafeCoerce $\lambda a \rightarrow (a, a)$

reclaim = unsafeCoerce $\lambda a \rightarrow a$

share = unsafeCoerce $\lambda a \rightarrow \text{Ur } a$

Case Study

Pure Borrow is Already There!

SoftwareFoundationGroupAtKyotoU / pure-borrow

Public

Notifications

Fork 0

Star 61

<> Code

Issues

Pull requests 1

Actions

Projects

Security and quality

Insights

main

Go to file

<> Code

konn

Merge pull request #41 from SoftwareFoundat...

24fabd7 · last month

.agents/hooks	chore(agents): share Claude Cod...	2 months ago
.claude	chore(agents): share Claude Cod...	2 months ago
.codex	refactor(concurrent): rewrite qso...	last month
.github/workflows	ci: upgrade run-fourmolu to v13 a...	last month
.vscode	Suppressing further	9 months ago
app	chore(bench): omit missing case...	last month
bench	review: document the delimiter o...	last month
ci	Convinced GHC 9.10	5 months ago
doctests	fourmolized	5 months ago
internal-src	perf(worklist): add checked-surf...	last month
scripts	Auto-plot	5 months ago

About

Pure Borrow: Linear Haskell Meets Rust-Style Borrowing

Readme

BSD-3-Clause license

Activity

Custom properties

61 stars

11 watching

0 forks

Report repository

Releases 1

Release 0.0.0.0

Latest

4 months ago

Contributors 4

konn

Hiromi Ishii

shiatsumat

Yusuke Matsu...

pure-borrow: Rust-style borrowing in Linear Haskell with purity

[[bsd3](#), [library](#), [linear-haskell](#), [program](#)] [[Propose Tags](#)] [[Report a vulnerability](#)]

This package realizes rust-style borrowing in Linear Haskell with purity and concurrency support. See [Control.Monad.Borrow.Pure](#) for the main API documentation, and see our paper *Pure Borrowing: Linear Haskell Meets Rust-Style Borrowing* by Y. Matsushita and H. Ishii for the details.

[\[Skip to Readme\]](#)

Build

BuildFailed

Documentation

Available

Modules

[\[Index\]](#) [\[Quick Jump\]](#)

Control

Concurrent

DivideConquer

Control.Concurrent.DivideConquer.Linear

STM

Control.Concurrent.STM.TMDeque

Control.Concurrent.STM.TMDequeRingBuffer

Monad

Borrow

Versions [\[RSS\]](#)

0.0.0.0

Change log

[CHANGELOG.md](#)

Dependencies

[array](#), [base](#) ([>=4.17](#) & [<5](#)), [bytestring](#), [cassava](#), [containers](#), [deepseq](#), [directory](#), [file-embed](#), [hybrid-vectors](#), [linear-base](#) ([>=0.7](#)), [linear-generics](#), [monoidal-containers](#), [optparse-applicative](#), [process](#), [pure-borrow](#), [random](#), [stm](#), [tasty](#), [tasty-bench](#), [template-haskell](#), [temporary](#), [text](#), [transformers](#), [unordered-containers](#), [vector](#), [vector-algorithms](#) [\[details\]](#)

Tested with

[ghc](#) [==9.10.2](#) || [==9.12.4](#) || [==9.14.1](#)

License

[BSD-3-Clause](#)

Copyright

Copyright (c) 2025-present, Yusuke Matsushita and Hiromi Ishii

Author

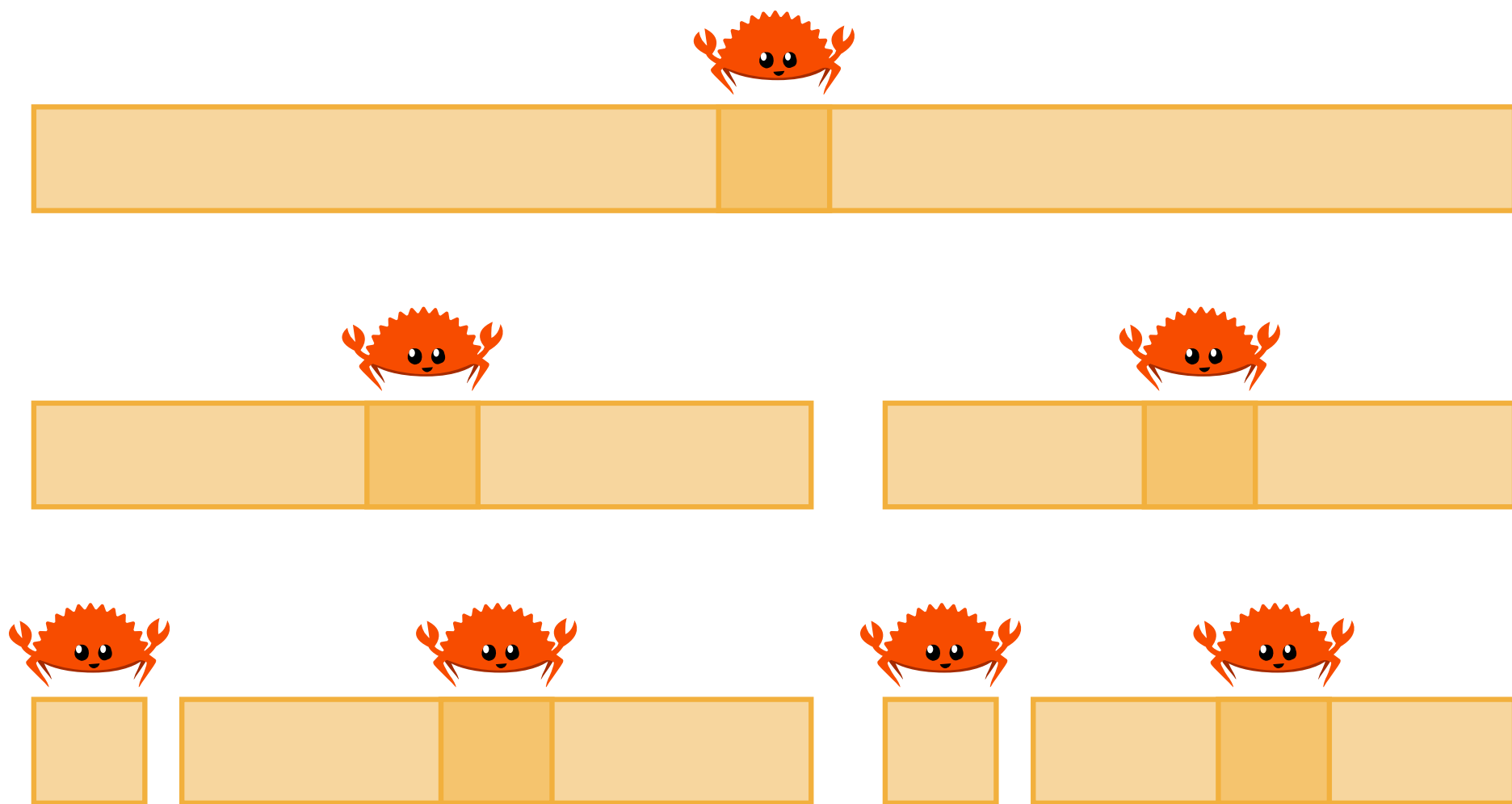
Yusuke Matsushita and Hiromi Ishii

Pure Borrow works on stable *GHC*

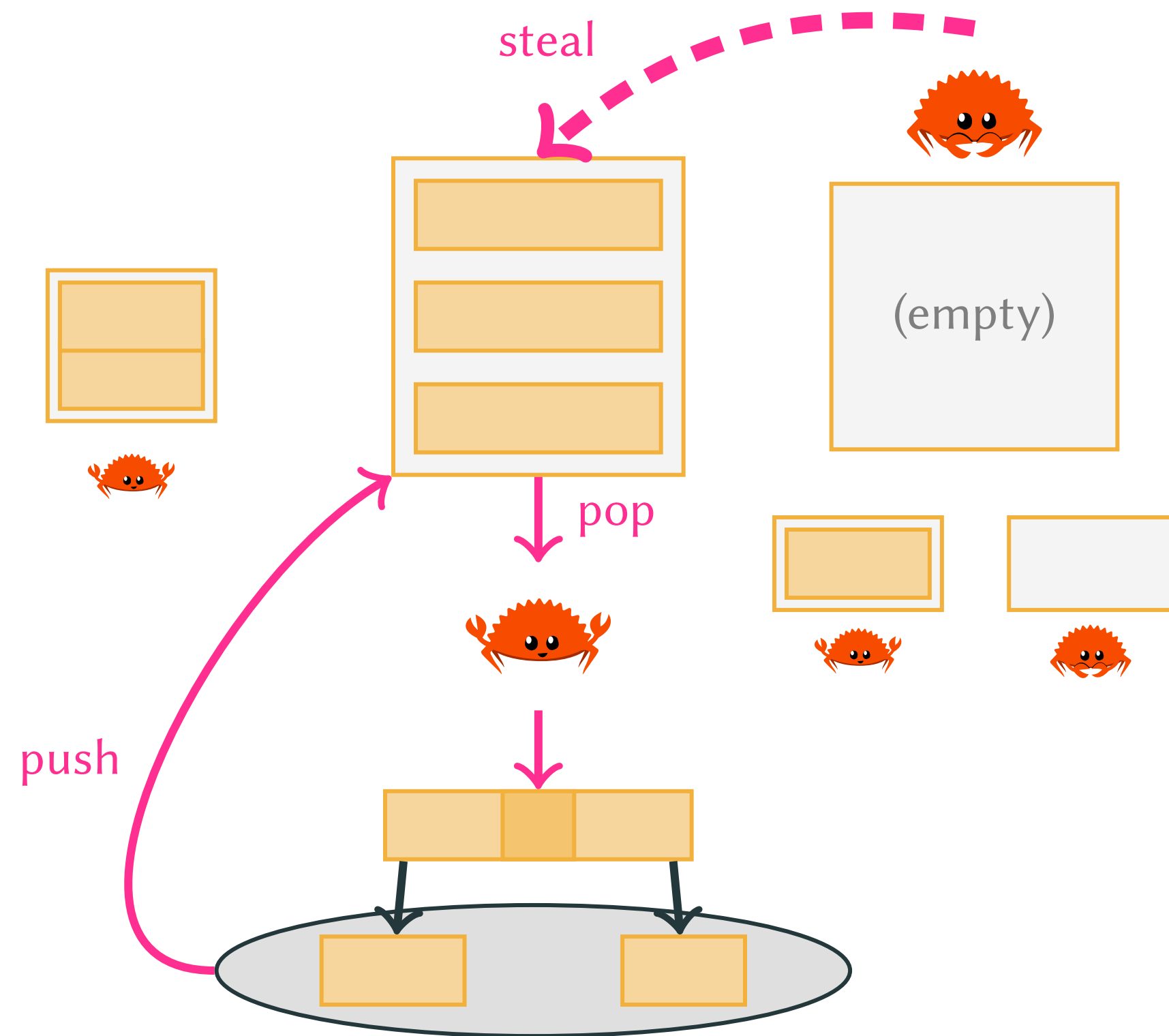
Parallel Quicksort

Pure + *Mutate* + *Parallel*

```
qsort :: Mutα (Vec Int) → BOα ()
qsort v = let (Ur n, v) = size v in
  if n ≤ 1 then pure (consume v) else do
    (Ur a, v) ← copyAt v (n // 2)
    (i, v) ← pivotSwap v a
    let (lo, hi) = splitAt v i
    consume <$> parBO (qsort lo) (qsort hi)
```



Advanced: Work-Stealing Scheduler

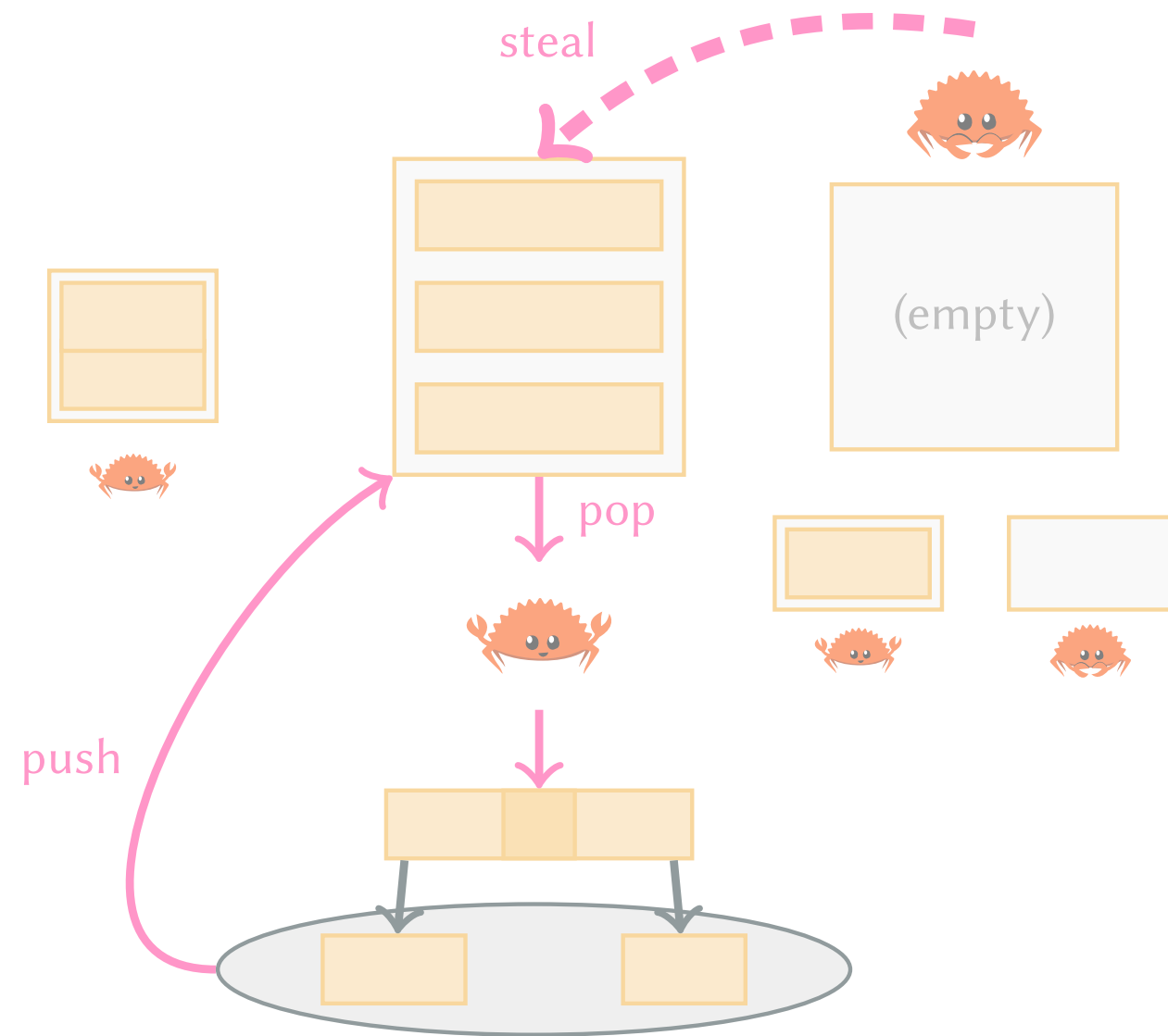


data DivideConquer ^{α} a = DivideConquer
($\forall \beta. \beta \leq \alpha \Rightarrow \text{Mut}^\beta \text{ a} \multimap \text{BO}^\beta (\text{Res}^\beta \text{ a})$)

data Res ^{β} a = Done | Cont [$\text{Mut}^\beta \text{ a}$]

divideAndConquer :: $\beta \leq \alpha \Rightarrow$
Int $\rightarrow$ DivideConquer ^{α} a $\rightarrow$
 $\text{Mut}^\alpha \text{ a} \multimap \text{BO}^\beta (\text{Mut}^\alpha \text{ a})$

Parallel Quicksort by Work Stealing



data DivideConquer^α a = DivideConquer
 (∀ β. β ≤ α ⇒ Mut^β a ⇝ BO^β (Res^β a))

data Res^β a = Done | Cont [Mut^β a]

divideAndConquer :: β ≤ α ⇒
 Int → DivideConquer^α a →
 Mut^α a ⇝ BO^β (Mut^α a)

qsortDC :: β ≤ α ⇒ Int →

Mut^α (Vec Int) ⇝ BO^β (Mut^α (Vector Int))

qsortDC nwork = divideAndConquer nwork \$

DivideConquer λv → **case** size v **of**

(Ur n, v)

| n ≤ 1 → **let** () = consume v **in** pure Done

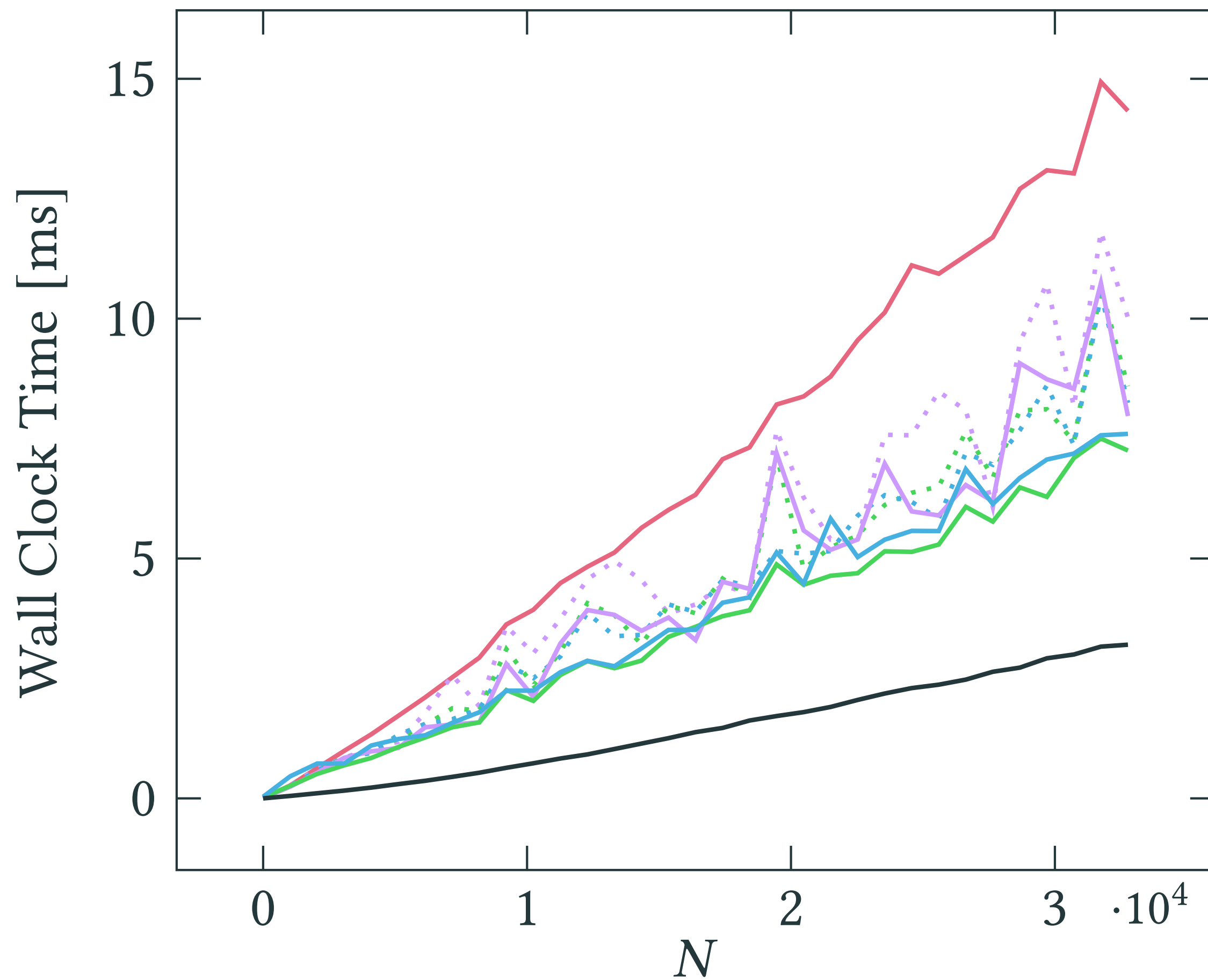
| otherwise →

(Ur a, v) ← copyAtMut v (n // 2)

(lo, hi) ← pivotSwap v a; pure \$ Cont [lo; hi]

No thread joining needed → **Efficient**

Benchmark Results

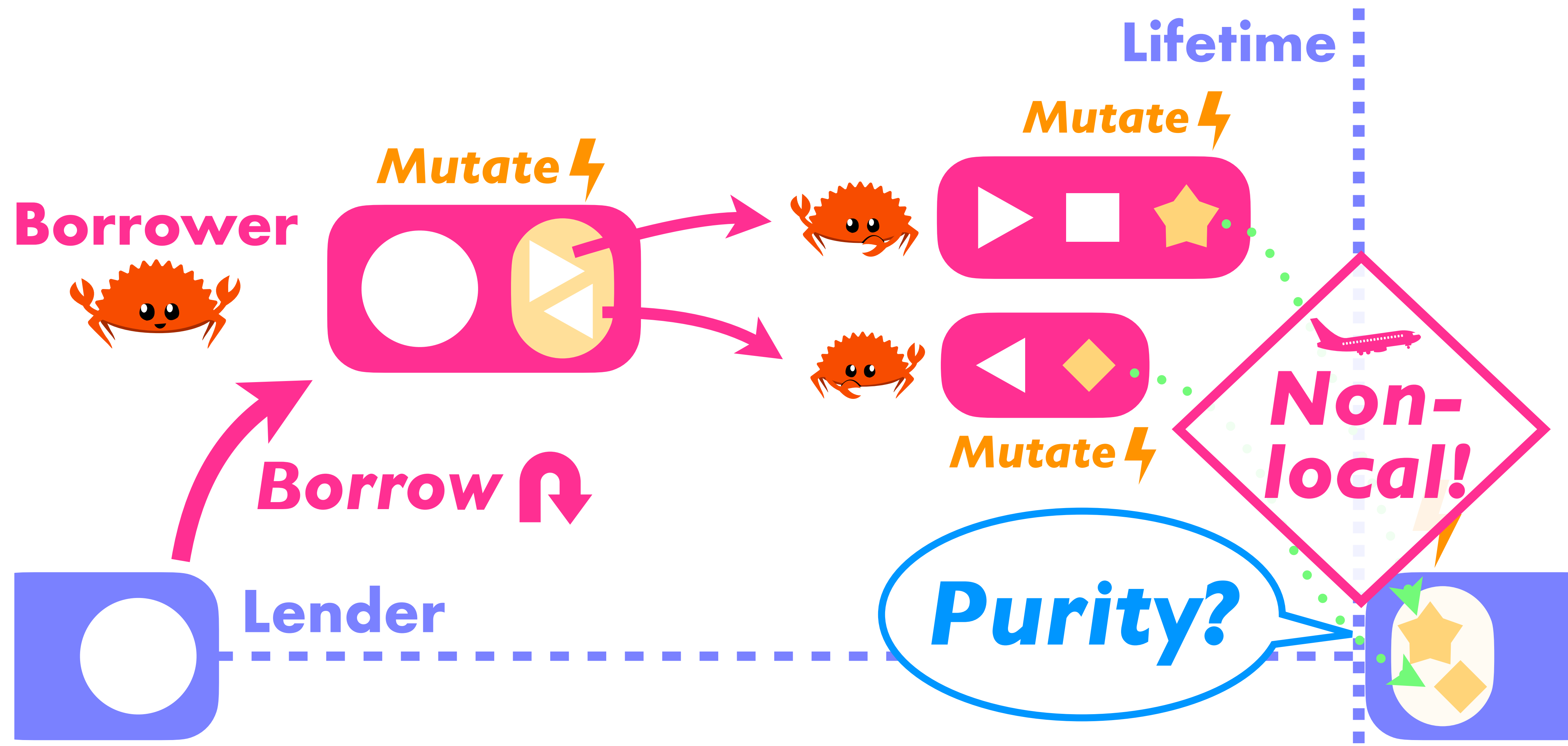


***Successfully
parallelized***

— Sequential Naïve 4 Naïve 16 Naïve 32
— WS 4 — WS 8 — WS 10 — Industrial

Metatheory

Challenge: Purity of Borrowing



Challenge: Purity of Borrowing

Borrow $\text{borrow} :: \text{Lin} \Rightarrow a \multimap (\text{Mut}^\alpha a, \text{Lend}^\alpha a)$

Mutate $\text{modifyAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap$
 $\text{Int} \rightarrow (a \multimap a) \multimap \text{BO}^\alpha (\text{Mut}^\alpha (\text{Vec } a))$

Split $\text{splitAt} :: \text{Mut}^\alpha (\text{Vec } a) \multimap$
 $\text{Int} \rightarrow (\text{Mut}^\alpha (\text{Vec } a), \text{Mut}^\alpha (\text{Vec } a))$

Reclaim $\text{reclaim} :: \text{End}^\alpha \Rightarrow \text{Lend}^\alpha a \multimap a$

Purity?



Idea: Histories

- ✦ Manage the global **history** of **mutations** to **borrow**s for each **lifetime**
- ✦ **BO** monad: **State** monad with the **history** state
- ✦ **Lifetime end** class: Owns frozen **history**
- ✦ **reclaim**: Restore the **updated** value from the **history**

Our Formalization & Metatheory

- ✦ Core calculus + Linear type system for Pure Borrow
- ✦ Two operational semantics
 - ▶ Mutative, with memories & Denotational, based on histories
- ✦ Theorem: Denotational op. sem. is confluent
 - ▶ Full purity: Future work, due to fresh borrow id generation
- ✦ WIP: Mutative & denotational opsems bisimulate?
 - ▶ For well-typed programs & configurations

Reference Core Calculus Syntax

Variable	$Var \ni x, y, z$	Data constructor	C
Operator	$op ::= (+) \ 2 \mid \cdots \mid (\leq) \ 2 \mid \cdots$ $\mid \text{par} \ 2 \mid \text{consume} \ 1 \mid \text{move} \ 1$ $\mid \text{lin} \ 1 \mid \text{withLin} \ 1 \mid \text{newRef} \ 2 \mid \text{freeRef} \ 1$ $\mid \text{newLifetime} \ 2 \mid \text{endLifetime} \ 1 \mid \text{execBO} \ 2$ $\mid \text{borrow} \ 2 \mid \text{share} \ 1 \mid \text{copy} \ 1 \mid \text{joinMut} \ 1 \mid \text{reclaim} \ 2$		
Monad constructor	$mo ::= \text{pure} \ 1 \mid (\gg=) \ 2 \mid \text{sexecBO} \ 2$ $\mid \text{parBO} \ 2 \mid \text{deref} \ 1 \mid \text{updateRef} \ 2$		
Term	$Term \ni t, u ::= x \mid \text{let } \overline{x = t} \text{ in } u \mid \lambda x \rightarrow t \mid t \ u \mid \text{seq} \ x \ t \mid n$ $\mid C \ \bar{t} \mid \text{case } t \text{ of } \{ \overline{C \ \bar{x} \rightarrow u} \} \mid op \ \bar{t} \mid mo \ \bar{t}$		

Reference Core Calculus Type Syntax

Lifetime variable α, β Lifetime id ι

Lifetime $\alpha, \beta ::= \text{al } \iota \mid \text{static} \mid \alpha \wedge \beta \mid \alpha$

Multiplicity variable π, μ Multiplicity $\pi, \mu ::= 1 \mid \omega \mid \prod \bar{\pi}$

Type variable X, Y Variable $x ::= \alpha \mid \iota \mid \pi \mid X$

Type constructor F Borrower constructor $B ::= \text{Mut} \mid \text{Share}$

Type $Type \ni T, U ::= \forall x. T \mid X \mid T \rightarrow_{\pi} U \mid F \bar{T} \mid \text{Int} \mid \text{Lin}$
 $\mid \text{Ref } T \mid \text{Now}^{\alpha} \mid \text{End}^{\alpha} \mid B^{\alpha} T \mid \text{Lend}^{\alpha} T \mid \text{BO}^{\alpha} T$

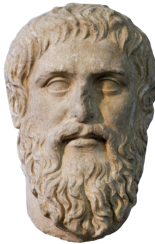
$T \multimap U \triangleq T \rightarrow_1 U$ $T \rightarrow U \triangleq T \rightarrow_{\omega} U$

Data type declaration **data** $F \bar{X}$ **where** $\overline{C :: T \rightarrow_{\pi} F \bar{X}}$

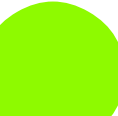
Typing context $\Gamma, \Delta ::= \overline{x ::_{\pi} T}$

Related Work

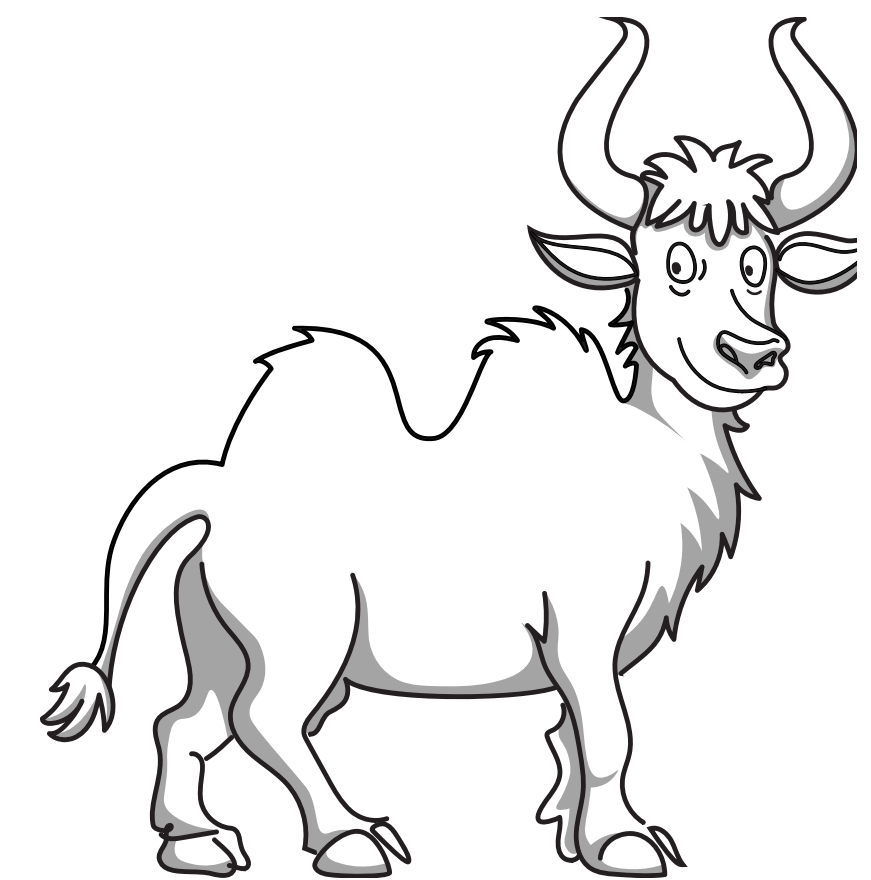
Recap High-Level Comparison

	Pure 	Mutate ⚡ Borrow ↻	Parallel	Leak-free
Pure Only	●	○	●	
ST Monad	●	●	○	○
Rust	○	●	●	○
Basic Linear Haskell	●	●	●	●
Pure Borrow	●	●	●	●

Comparison with Monads

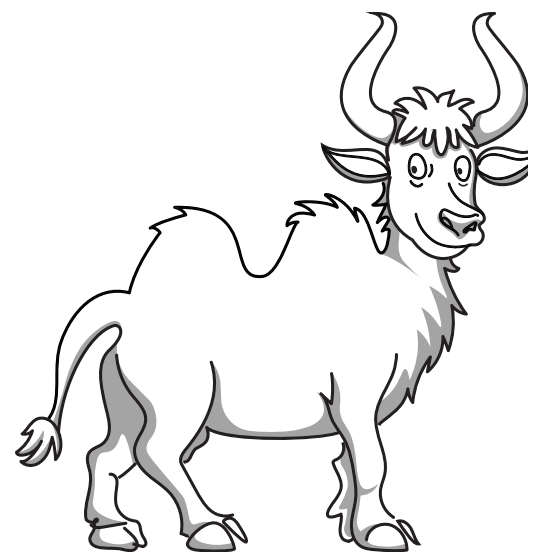
	Pure	Mutate	Borrow	Scope	Parallel	Leak-free
IO Monad <i>Peyton Jones+ '93</i>						
ST Monad <i>Launchbury+ '94</i>						
Monadic Regions <i>Fluet+ '04</i>						
RIO Monad <i>Bernardy+ '18</i>						
Pure Borrow						

- ✦ **OCaml** + **Safe low-level** control
 - ▶ **Ownership**, **memory-usage**, **thread safety**, ...
- ✦ **Lightweight mode** annotation
 - ▶ Easy to add to legacy OCaml code
 - ▶ Coarse **lifetime** modes: **global** vs **local**
- ✦ **No** strict **purity**



Linearity vs Uniqueness

Marshall+ '22



Linearity

Uniqueness

Comonad

many

$\leq$

once

unique

$\leq$

aliased
Monad



Non-linearity Ur

Comonad

$Ur\ a \multimap a$

$Ur\ a \multimap Ur\ (Ur\ a)$

Time-delimited aliasing Mut^α & $Lend^\alpha$

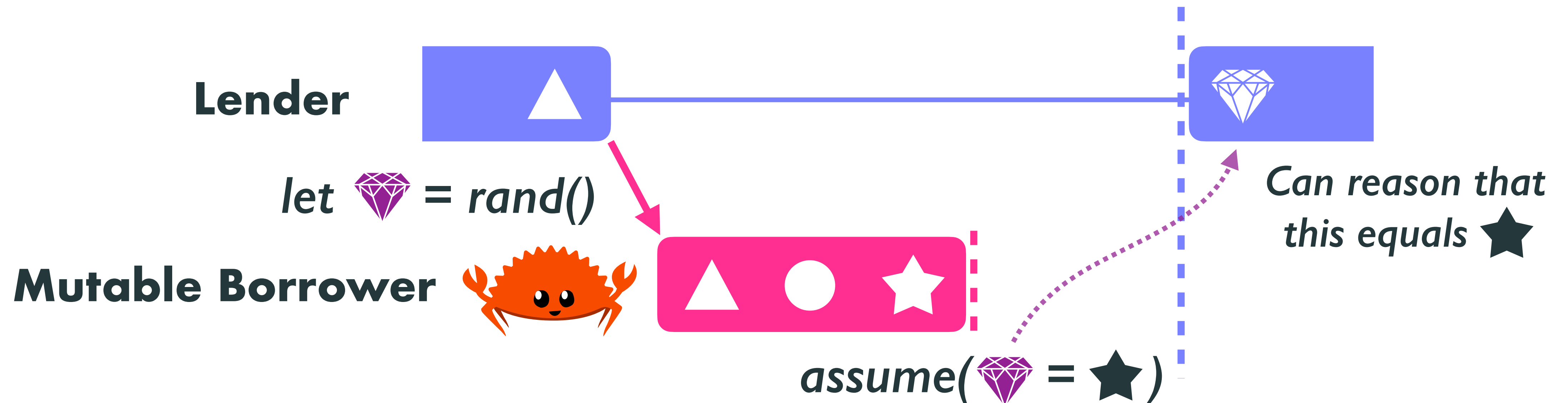
Monad-like

$a \multimap (Mut^\alpha\ a, Lend^\alpha\ a)$

$Mut^\alpha\ (Mut^\beta\ a) \multimap Mut^{\alpha \wedge \beta}\ a$

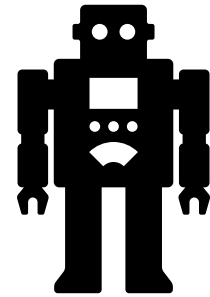
- ✦ Verify Rust programs as **functional** code
- ✦ Model **mutable borrows** by **prophecy** 
 - Prophecy [Abadi+ '88]: Fetches **info** about the **future**

*Final result of the
borrower's mutation*



Example of RustHorn'' Translation

Rust



Functional



```
fn take_max< $\alpha$ >  
  (ma: & $\alpha$  mut i32, mb: & $\alpha$  mut i32)  
  -> & $\alpha$  mut i32 {  
    if *ma >= *mb { ma } else { mb }  
  }
```

```
fn test(mut a: i32, mut b: i32) {  
  *take_max(&mut a, &mut b) += 7;  
  assert!((a - b).abs() >= 7);  
}
```

```
let take_max (a, a') (b, b') =   
  if a >= b then  
    (assume(b' = b); (a, a'))  
  else (assume(a' = a); (b, b'))
```

```
let test a b =  
  let a' = rand() in let b' = rand() in  
  let (r, r') =  
    take_max (a, a') (b, b') in  
  assume(r' = r + 7);  
  assert(abs (a' - b') >= 7);
```

Prophecies vs Histories

Prophecies

- ◆ **Concise & local**
 - ▶ Ideal for verification
- ◆ **Non-determinism**
- ◆ **Precise resolution needed**
 - ▶ Otherwise over-approximate

Histories

- ◆ **Rather bulky & global**
- ◆ **Deterministic**
- ◆ **Robust**
 - ▶ Ideal for the purity proof

Conclusion

Summary: Pure Borrow

- ✦ **Linear Haskell + Rust-style borrowing**
 - ▶ **Pure** & **Mutate** w/o ownership passing & **Parallelize**
 - ▶ **BO** monad for **lifetime** constraints
- ✦ Case study: **Parallel** quicksort
- ✦ **Metatheory**: **Purity** by **history**-based model



Future Work

- ✦ **Logical relations** like RustBelt [Jung+ '18] ?
 - ▶ Using **separation logic** [O'Hearn+ '01] such as Iris [Jung+ '15] ?
- ✦ **Denotational semantics?**
 - ▶ Fresh borrow id generation, as in ν -calculus [Pitts+ '93]
 - ▶ **Purity & Equational theory?**
 - ▶ Prove soundness? Using logical relations?
- ✦ **BO monad transformer?**

Thank you!

